File Management

Files

- Files are the central element to most applications
  - Input to applications is by means of a file
  - Output is saved in a file for long-term storage
- The File System is one of the most important part of the OS to a user
  - It provides the resource abstractions with secondary storage
- Desirable properties of files:
  - Long-term existence: do not disappear at log off
  - Sharable between processes: files have names and permissions
  - Structure: internal structure + hierarchical structure reflecting relationships among files

File Management

- File Management System consists of system utility programs that run as privileged applications
- Concerned with secondary storage
- FMS provide functions performed on files, e.g.:
  - Create: A new file is defined and positioned within the structure of files
  - Delete: A file is removed from the file structure and destroyed
  - Open: allowing the process to perform functions on the file
  - Close: The file is closed w.r.t a process, so that the process no longer may perform functions on the file, until the process opens the file again.
  - Read: A process reads all or a portion of the data in a file.
  - Write: A process updates a file, either by adding new data that expands the size of the file or by changing the values of existing data items in the file.
- FMS provides services to users and applications in the use of files
  - The only way a user or application accesses files
- Programmer does not need to develop file management software

Objectives for File Systems

- Meet the data management needs of the user
- Guarantee that the data in the file are valid
- Optimize performance: throughput (system), response time (user)
- Provide I/O support for a variety of storage device types
- Minimize lost or destroyed data
- Provide a standardized set of I/O interface routines to user processes
- Provide I/O support for multiple users (if needed)

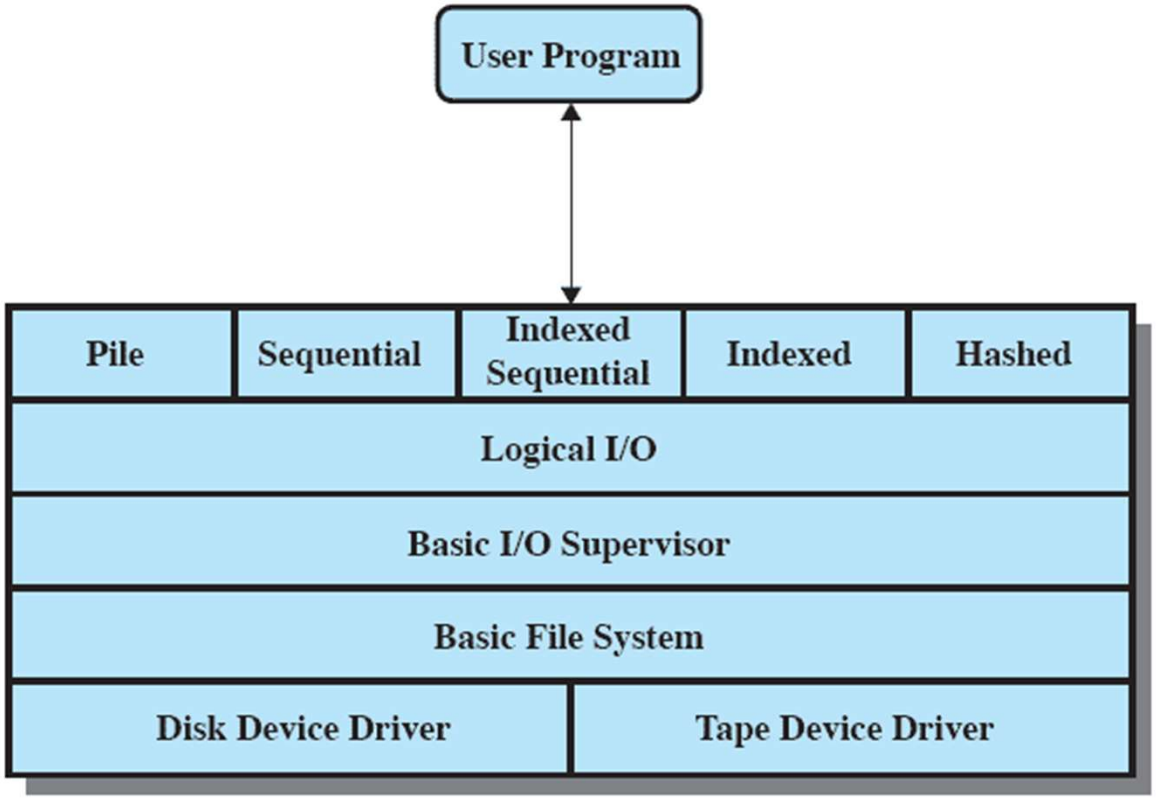
Minimum Requirements

- For an interactive, general-purpose system, the following constitute a minimal set of requirements:
  - A user should be able to create, delete, read, write, and modify files
  - A user may have controlled access to other users' files.
  - A user may control what types of accesses are allowed to his files
  - A user should be able to restructure his files in a form appropriate to the problem.
  - A user should be able to move data between files.
  - A user should be able to back up and recover the user's files in case of damage.
  - A user should be able to access his files by a symbolic name rather than by numeric identifier

Terms

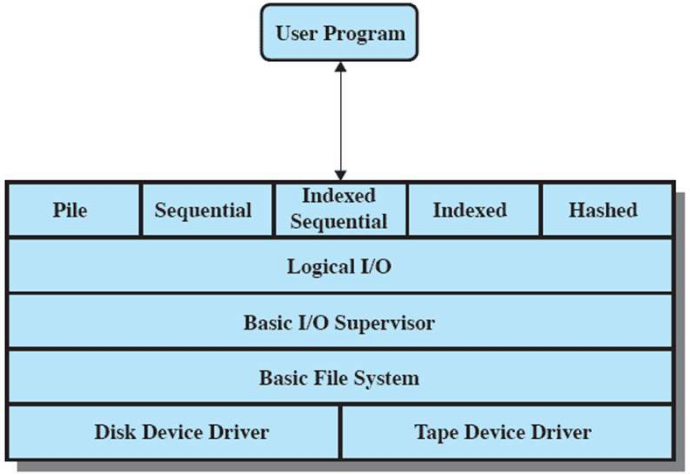
- **Fields**
  - Basic element of data, contains a single value (name, date,..)
  - Characterized by its length (fixed or variable) and data type
- **Records**
  - Collection of related fields treated as a unit (employee record)
  - May be of a fixed or variable size
- **File**
  - Collection of similar records
  - Treated as an entity by applications, Usually referenced by a name
  - Access control restrictions usually apply at the file level
- **Database**
  - Collection of related data
  - Relationships exist among elements
  - Consists of one or more files
  - Usually, there is a separate DBMS independent of the OS, although that system may make use of some file management programs

File System Architecture



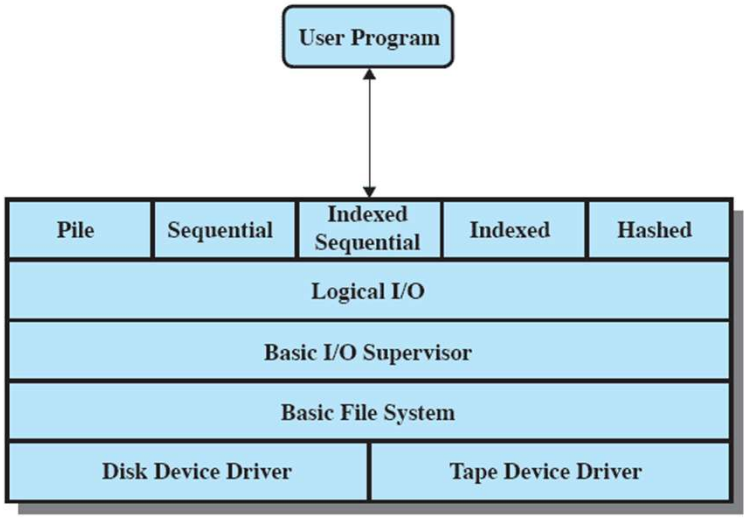
- Device Drivers Communicate directly with device
- Basic File System Buffering, placing data on device
- Basic I/O Supervisor I/O initiation and termination
- Logical I/O Deals with records
- Access method (sequential, hashed, …)
- Standard interface with the user

Device Drivers



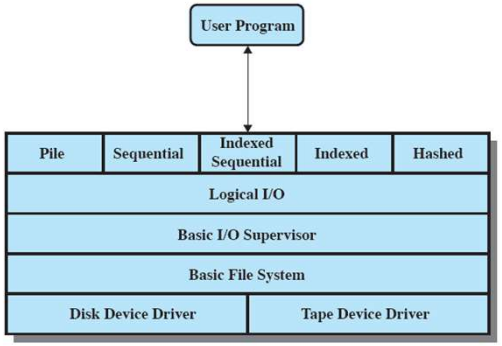
- Lowest level
- Communicates directly with peripheral devices, their controllers or channels
- Responsible for starting I/O operations on a device and processing the completion of an I/O request
- For file operations, the typical devices controlled are disk and tape drives.
- Device drivers are usually considered to be part of the operating system.

Basic File System or the physical I/O level



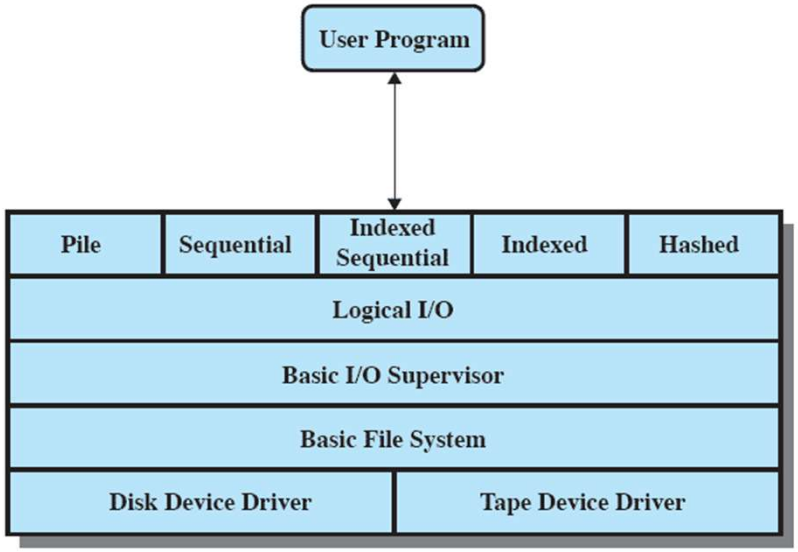
- Primary interface with the environment outside the computer system
- Deals with blocks of data exchanged with disk or tape
  - Concerned with the placement of blocks on storage device
  - Concerned with buffering those blocks in main memory
  - It does not understand the content of the data or the structure of the files involved.

Basic I/O Supervisor



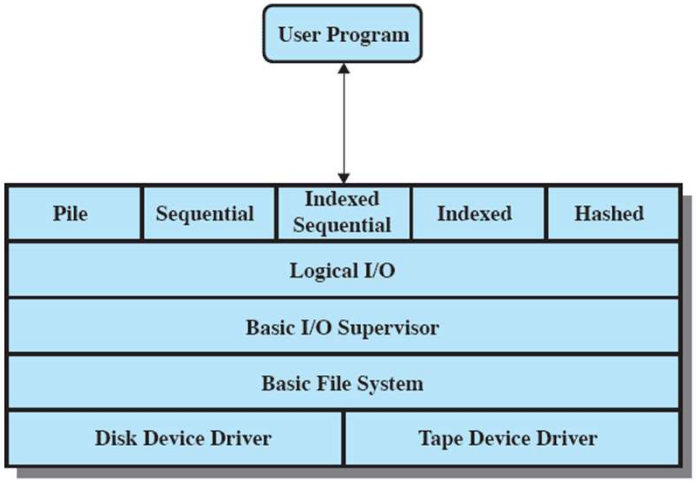
- Responsible for all file I/O initiation and termination.
- Control structures are maintained that deal with
  - Device I/O
  - Scheduling
  - File status
- Selects the device on which file I/O is to be performed
- schedules disk and tape I/O accesses to optimize performance: I/O buffers are assigned and secondary memory is allocated at this level

Logical I/O



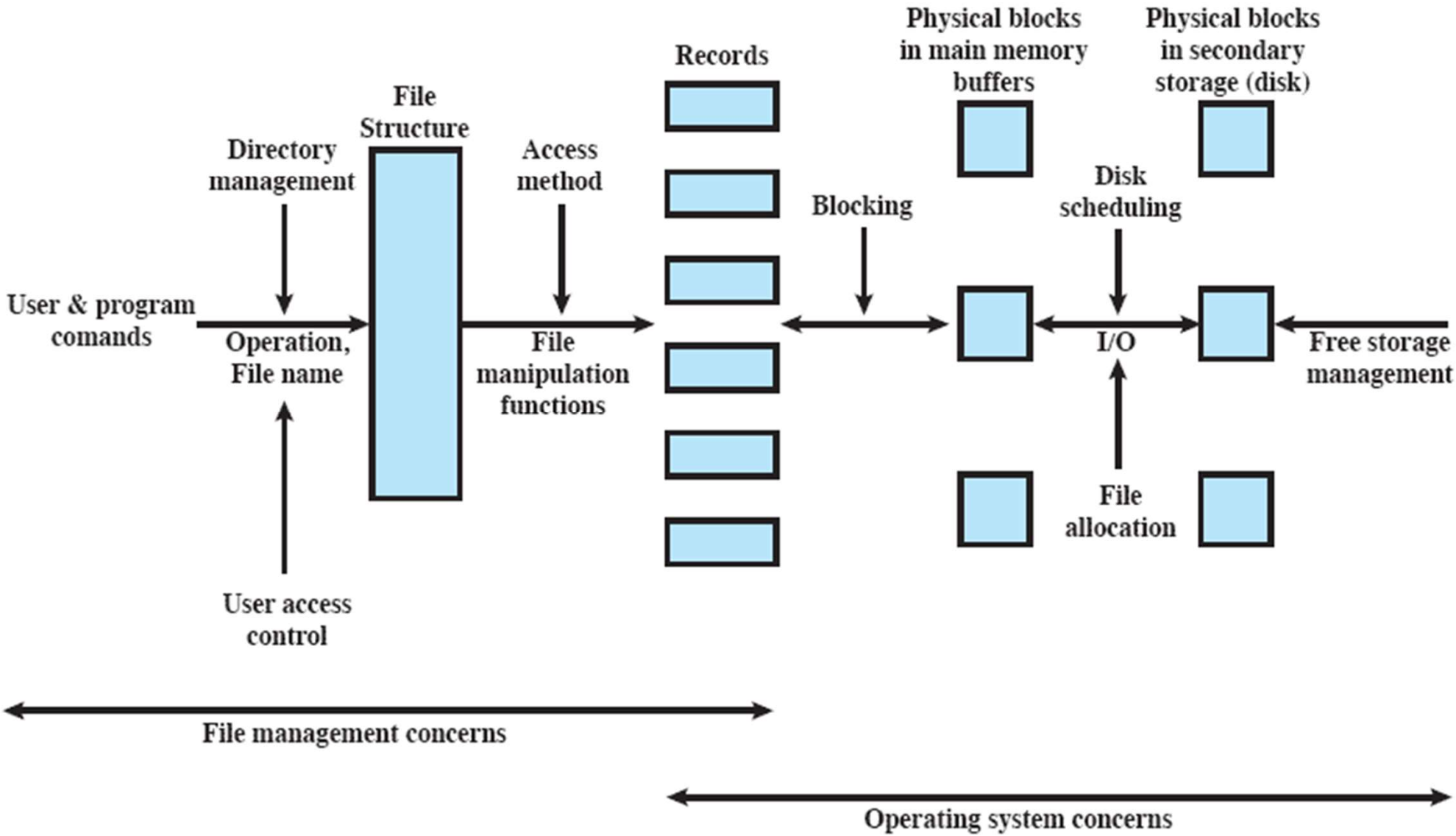
- Enables users and applications to access records
- The Basic File System deals with blocks of data while the logical I/O module deals with file records.
- Provides general-purpose record I/O capability
- Maintains basic data about files

Access Method



- Level closest to the user
- Provides a standard interface between applications and the file systems and devices that hold the data
- Reflect different file structures
- Different access methods reflect different file structures and different ways of accessing and processing the data

Elements of File Management



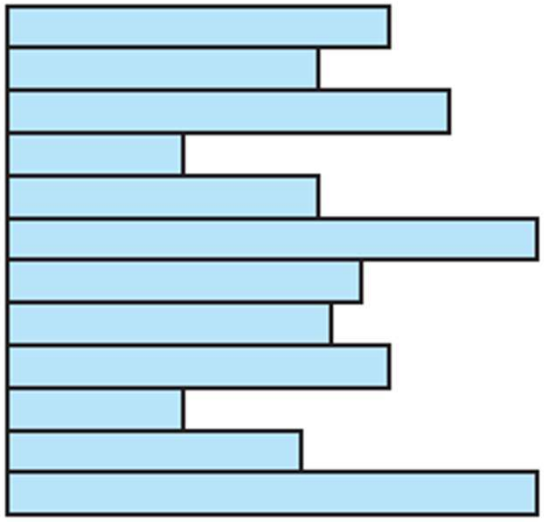
File Organization

- Refers to the **logical structure** of records
  - Physical organization discussed later
- Determines the ***way*** in which records are accessed
- **Criteria**:
  - Rapid access
  - Ease of update
  - Economy of storage
  - Simple maintenance
  - Reliability
- These criteria may vary in importance or conflict
  - CD-ROM – Ease of update irrelevant
  - Economy of storage – minimum redundancy
  - Redundancy – increase speed of access
  - Indexes – Faster but uses more storage

File Organisation Types

- Many exist, but usually variations of:
  - Pile
  - Sequential file
  - Indexed sequential file
  - Indexed file
  - Direct, or hashed, file

The Pile

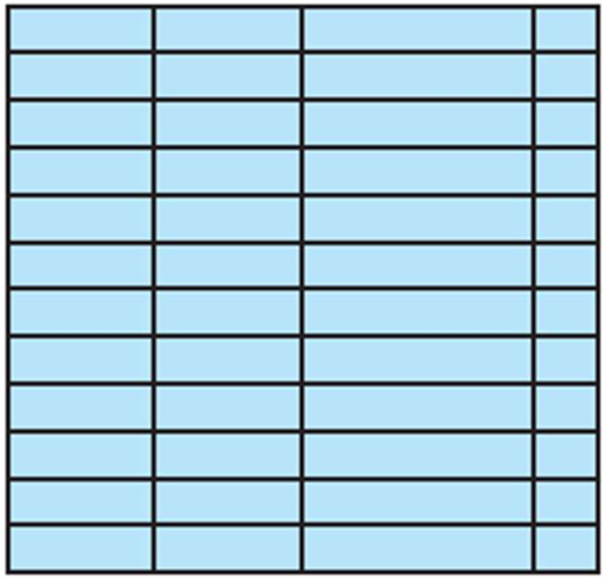


Variable-length records
Variable set of fields
Chronological order

- Purpose: accumulate a mass of data and save it
- Called a **heap** or a **pile** file.
- Data are collected in the order they arrive
- New records are inserted at the end of the file
  - ++ Easy to update
  - -- Record access is by exhaustive search
    - A **linear search** through the file records is necessary to search for a record: This requires reading and searching half the file blocks on the average, and is hence quite expensive.
    - Reading the records in order of a particular field requires sorting
- Records may be of different length ++ Uses space well
- Records may have different fields, or similar fields in different orders
  - -- Records must include not only value, but field name
  - -- The length of each field must be implicitly indicated by delimiters

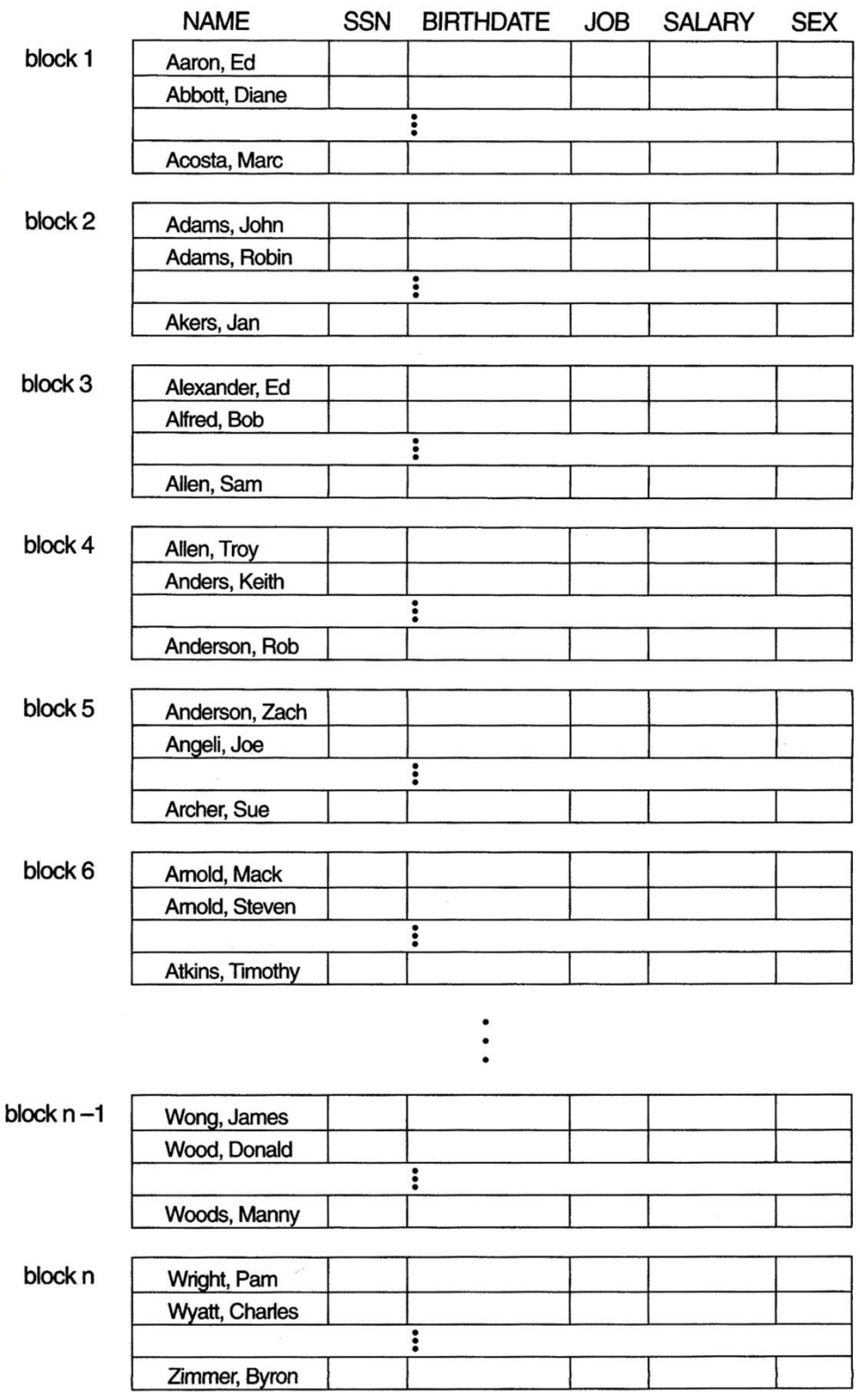
The Sequential File

- Fixed format used for records
- Records are the same length
- All fields the same (order and length)
- Only values of files need to be stored
- Field names and lengths are attributes of the file structure
- Key field
  - Uniquely identifies the record
  - Records are stored in key sequence



Fixed-length records
Fixed set of fields in fixed order
Sequential order based on key field

(b) Sequential File

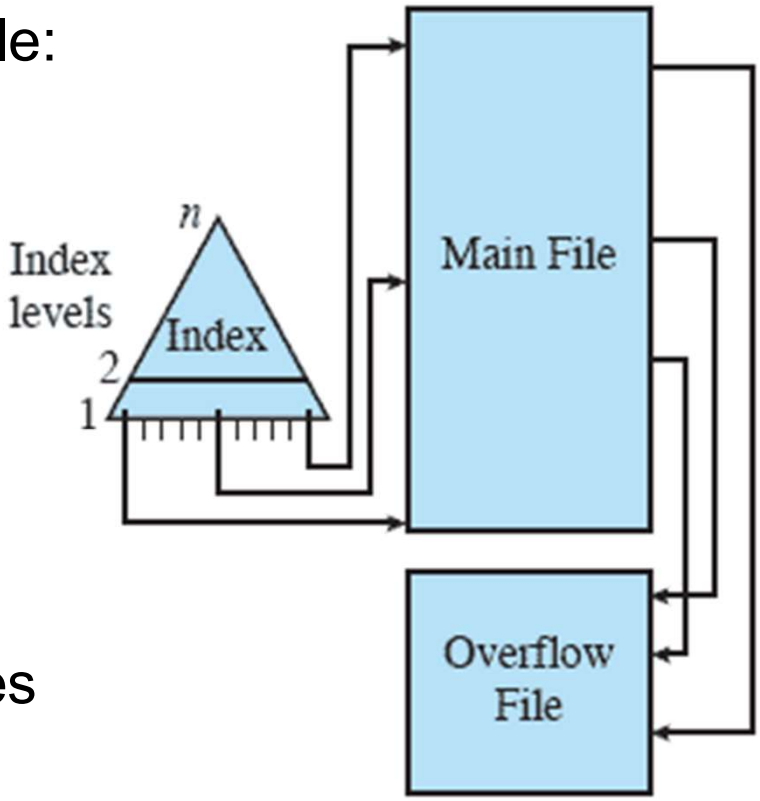

NAME
SSN
BIRTHDATE
JOB
SALARY
SEX
block 1
Aaron, Ed
Abbott, Diane
Acosta, Marc
block 2
Adams, John
Adams, Robin
Akers, Jan
block 3
Alexander, Ed
Alfred, Bob
Allen, Sam
block 4
Allen, Troy
Anders, Keith
Anderson, Rob
block 5
Anderson, Zach
Angeli, Joe
Archer, Sue
block 6
Arnold, Mack
Arnold, Steven
Atkins, Timothy
block n –1
Wong, James
Wood, Donald
Woods, Manny
block n
Wright, Pam
Wyatt, Charles
Zimmer, Byron

The Sequential File

- File records are kept sorted by the values of an *ordering field*.
- Reading the records in order of the ordering field is quite efficient.
- A **binary search** can be used to search for a record on its *ordering field* value.
  - This requires reading and searching $\log_2$ of the file blocks on the average, an improvement over linear search.
- Insertion is expensive: records must be inserted in the correct order
  - It is common to keep a separate unordered *overflow* (or *transaction*) file for new records to improve insertion efficiency; this is periodically merged with the main ordered file
- Handles random requests poorly
  - Must use sequential search

Indexed Sequential File

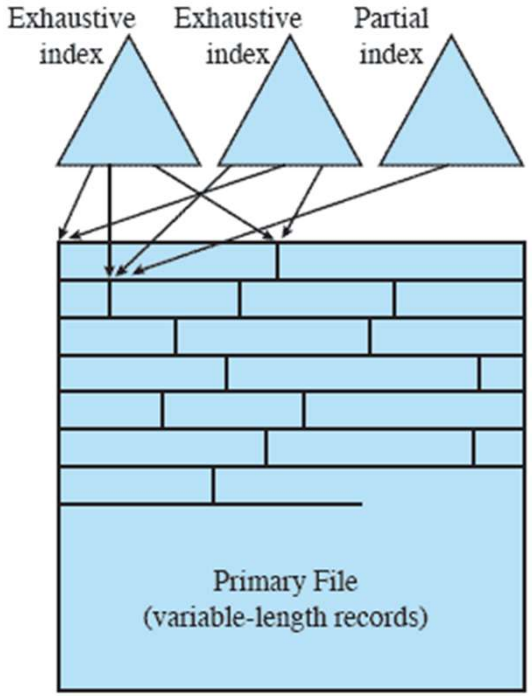
- Maintains the key ccs of the sequential file:
  - records are organized in sequence based on a key field.
- Two features are added:
  - an index to speed lookup
    - support random access
    - Index file is a sequential file
    - May have multiple levels of indexes
    - Limitation: effective processing is based on a single field: key
  - Overflow area to handle new records
    - Link from main records to overflow



(c) Indexed Sequential File

Indexed File

- concept of sequentiality is abandoned
- Records can be of variable length
- May have multiple indexes
  - One for each field we may search
- Records accessed only through indexes
- When a new record is added to the main file, all of the index files must be updated.
- Each index may be exhaustive or partial
  - An **exhaustive** (**dense) index** has an index entry for every search key value (and hence every record) in the data file.
  - A **partial (nondense, sparse) index**, on the other hand, has index entries for only some of the search values



(d) Indexed File

Indexes as Access Paths

- A single-level index is an auxiliary file that makes it more efficient to search for a record in the data file.
- The index is usually specified on one field of the file (although it could be specified on several fields)
- One form of an index is a file of entries **<field value, pointer to record>**, which is ordered by field value
- The index is called an access path on the field.
- The index file usually occupies considerably less disk blocks than the data file because its entries are much smaller
- A binary search on the index yields a pointer to the file record

Indexes as Access Paths (cont.)

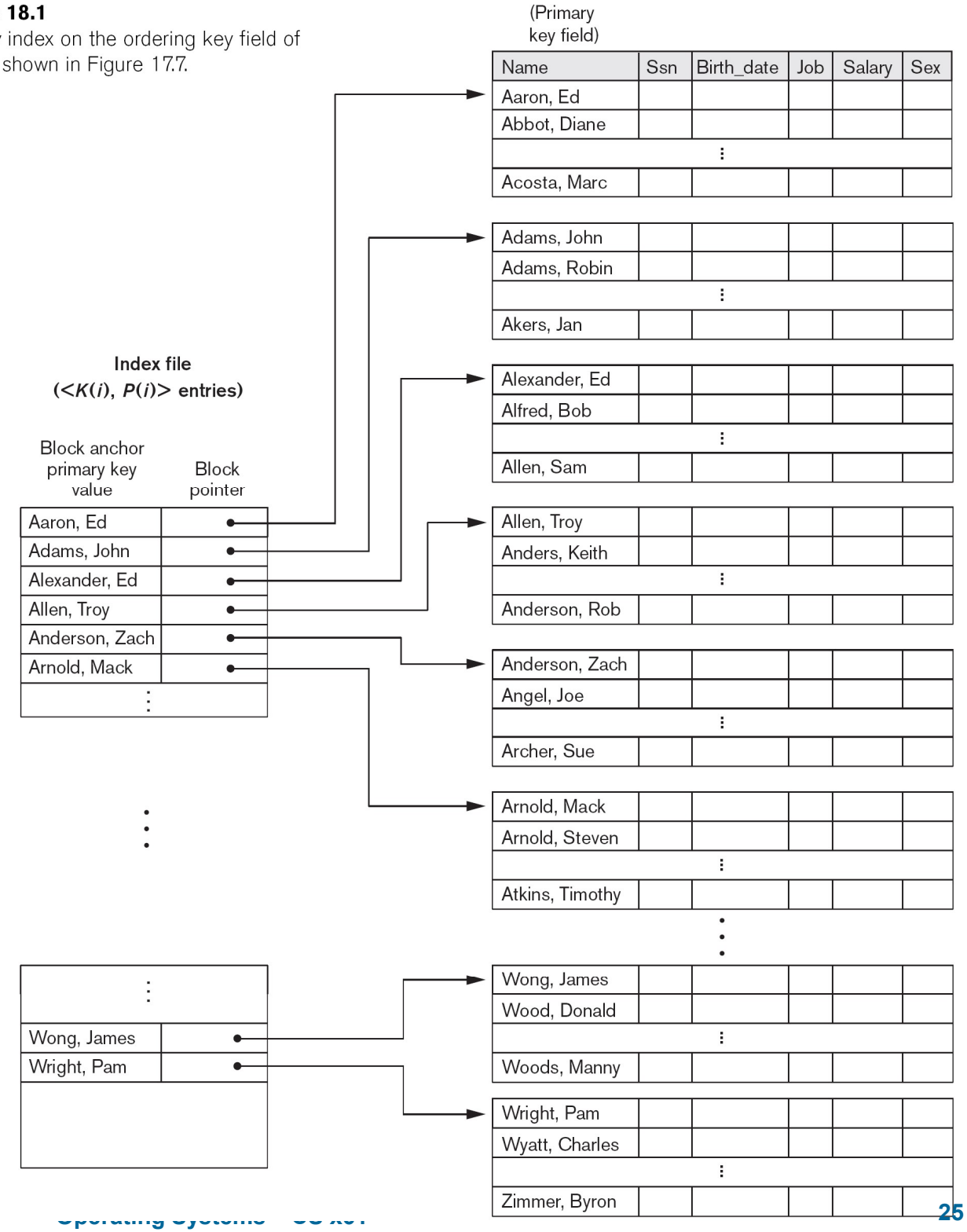
- Example: Given the following data file Employee(name, Ssn, Address, Job, Sal, ...)
- Suppose that:
  - record size R=150 bytes, block size B=512 bytes, r=30,000 records
- Then, we get:
  - blocking factor Bfr= B div R= 512 div 150= 3 records/block
  - number of file blocks b= (r/Bfr)= (30,000/3)= 10,000 blocks
- For an index on the SSN field, assume the field size V_{SSN}=9 bytes, assume the record pointer size P_R=7 bytes. Then:
  - index entry size $R_I=(V_{SSN}+ P_R)=(9+7)=16$ bytes
  - index blocking factor $Bfr_I= B \text{ div } R_I= 512 \text{ div } 16= 32$ entries/block
  - number of index blocks $b= (r/ Bfr_I)=$ (30,000/32)= 938 blocks
  - binary search needs $\log_2 bI= \log_2 938=$ 10 block accesses
  - This is compared to an average linear search cost of:
    - (b/2)= 10,000/2= 5,000 block accesses
  - If the file records are ordered, the binary search cost would be:
    - $\log_2 b=$ $\log_2 10{,}000=$ 14 block accesses

Types of Single-Level Indexes

- **Primary Index**
  - Defined on an ordered data file
  - The data file is ordered on a key field
  - Includes one index entry *for each block* in the data file; the index entry has the key field value for the *first record* in the block, which is called the *block anchor*
  - A similar scheme can use the *last record* in a block.
  - A primary index is a nondense (sparse) index, since it includes an entry for each disk block of the data file and the keys of its anchor record rather than for every search value.

Primary Index on the Ordering Key Field

Figure 18.1
Primary index on the ordering key field of the file shown in Figure 17.7.



Types of Single-Level Indexes

- **Clustering Index**
  - Defined on an ordered data file
  - The data file is ordered on a *non-key field* unlike primary index, which requires that the ordering field of the data file have a distinct value for each record.
  - Includes one index entry *for each distinct value* of the field; the index entry points to the first data block that contains records with that field value.
  - It is another example of *sparse* index where Insertion and Deletion is relatively straightforward with a clustering index.

A Clustering Index Example

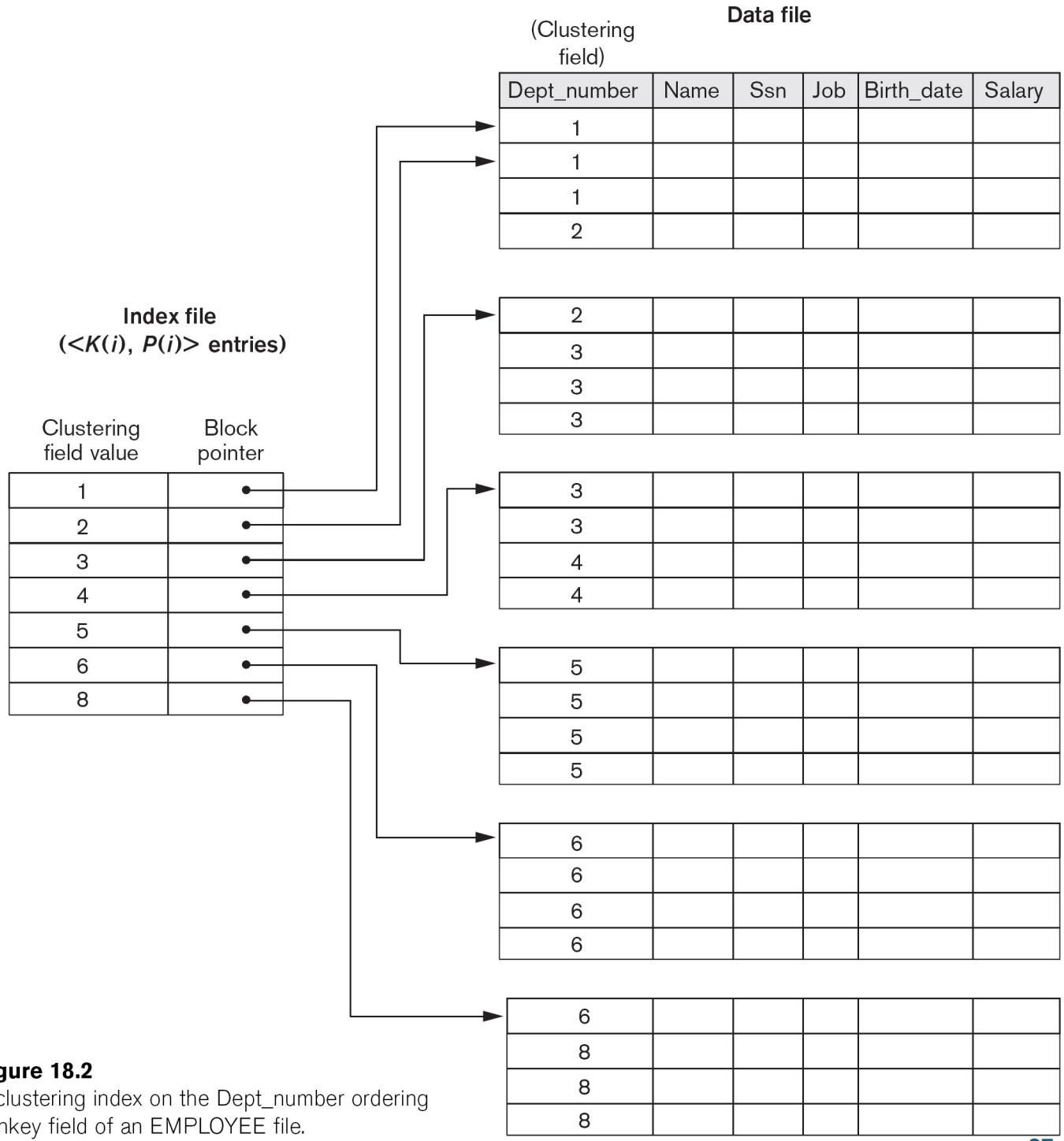


Figure 18.2
A clustering index on the Dept_number ordering nonkey field of an EMPLOYEE file.

Another Clustering Index Example

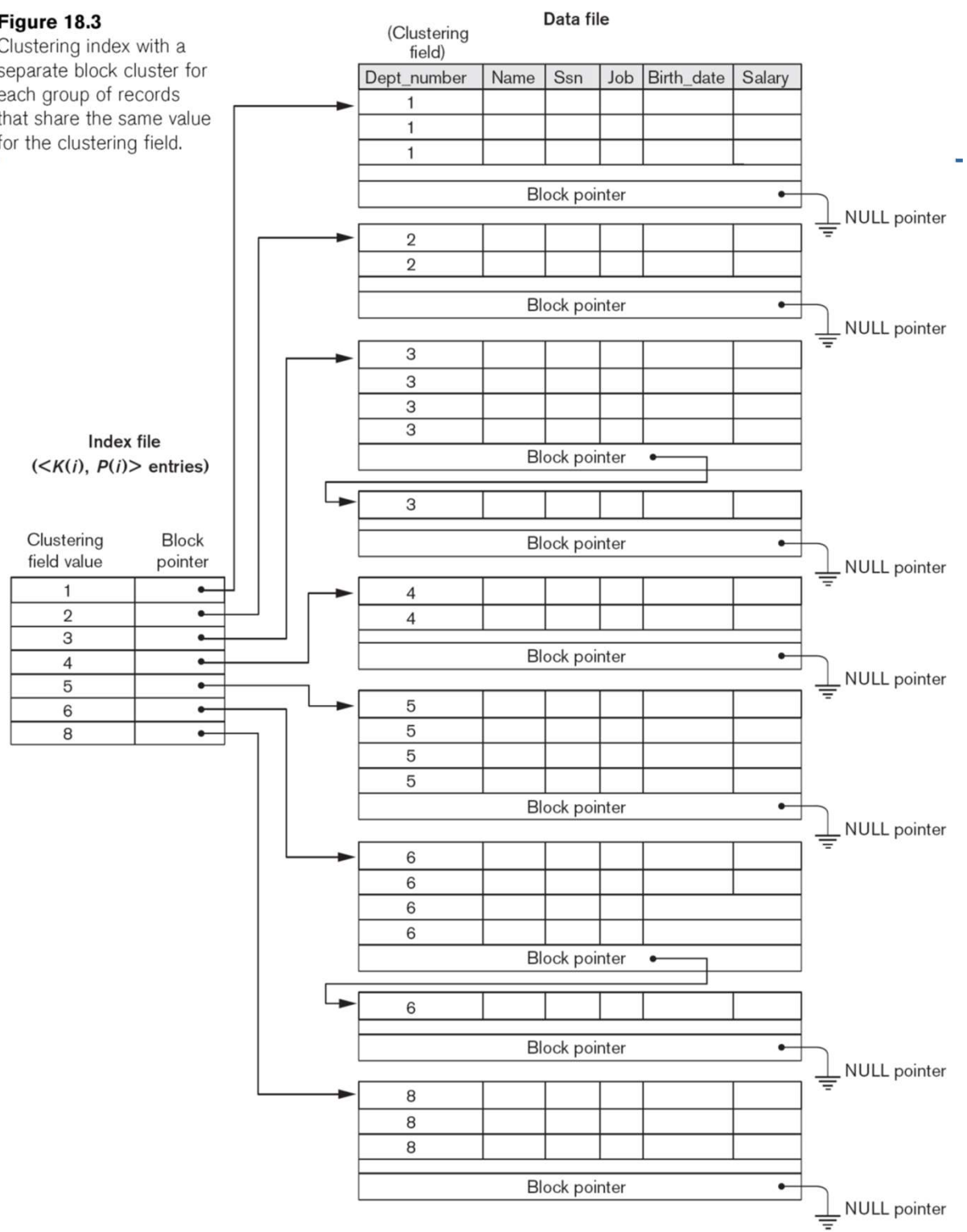


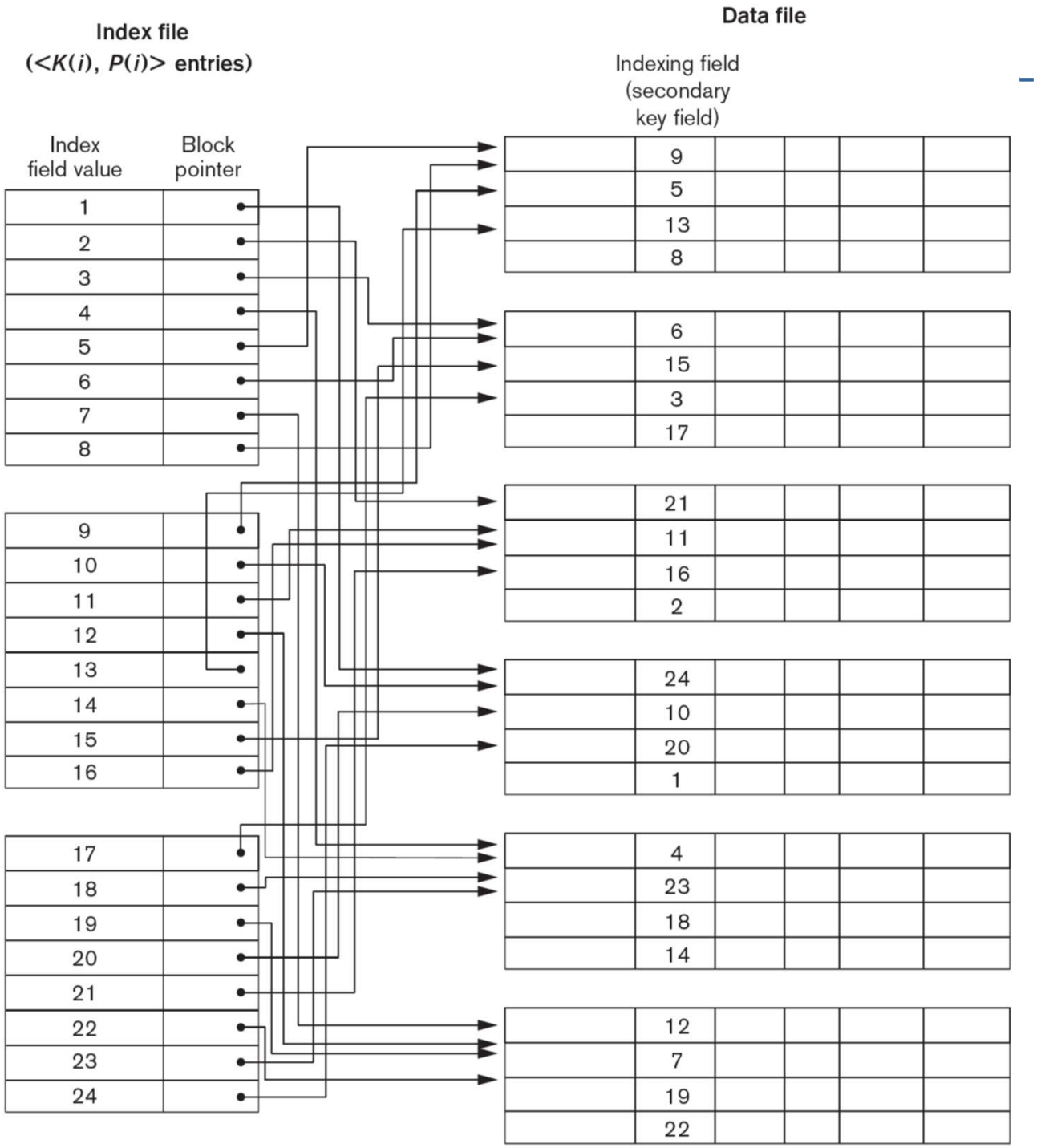
Figure 18.3
Clustering index with a separate block cluster for each group of records that share the same value for the clustering field.

Types of Single-Level Indexes

- **Secondary Index**
  - A secondary index provides a secondary means of accessing a file for which some primary access already exists.
  - The secondary index may be on a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.
  - The index is an ordered file with two fields.
    - The first field is of the same data type as some non-ordering field of the data file that is an **indexing field**.
    - The second field is either a **block** pointer **or** a **record** pointer.
  - There can be ***many*** secondary indexes (and hence, indexing fields) for the same file.
  - Includes one entry *for each record* in the data file; hence, it is a *dense index*

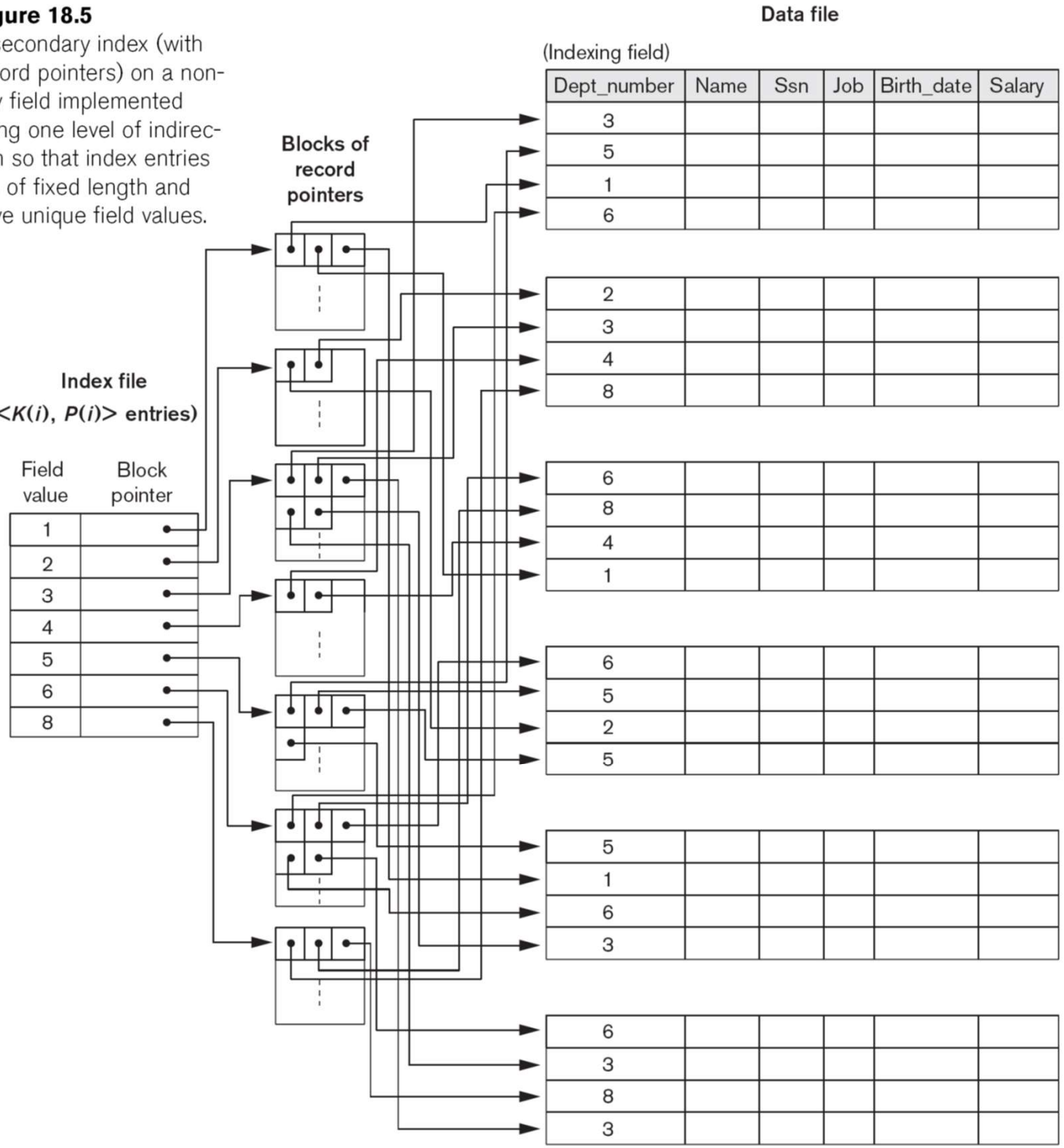
Example of a Dense Secondary Index

Figure 18.4
A dense secondary index (with block pointers) on a nonordering key field of a file.



Example of a Secondary Index

Figure 18.5
A secondary index (with record pointers) on a non-key field implemented using one level of indirection so that index entries are of fixed length and have unique field values.



Properties of Index Types

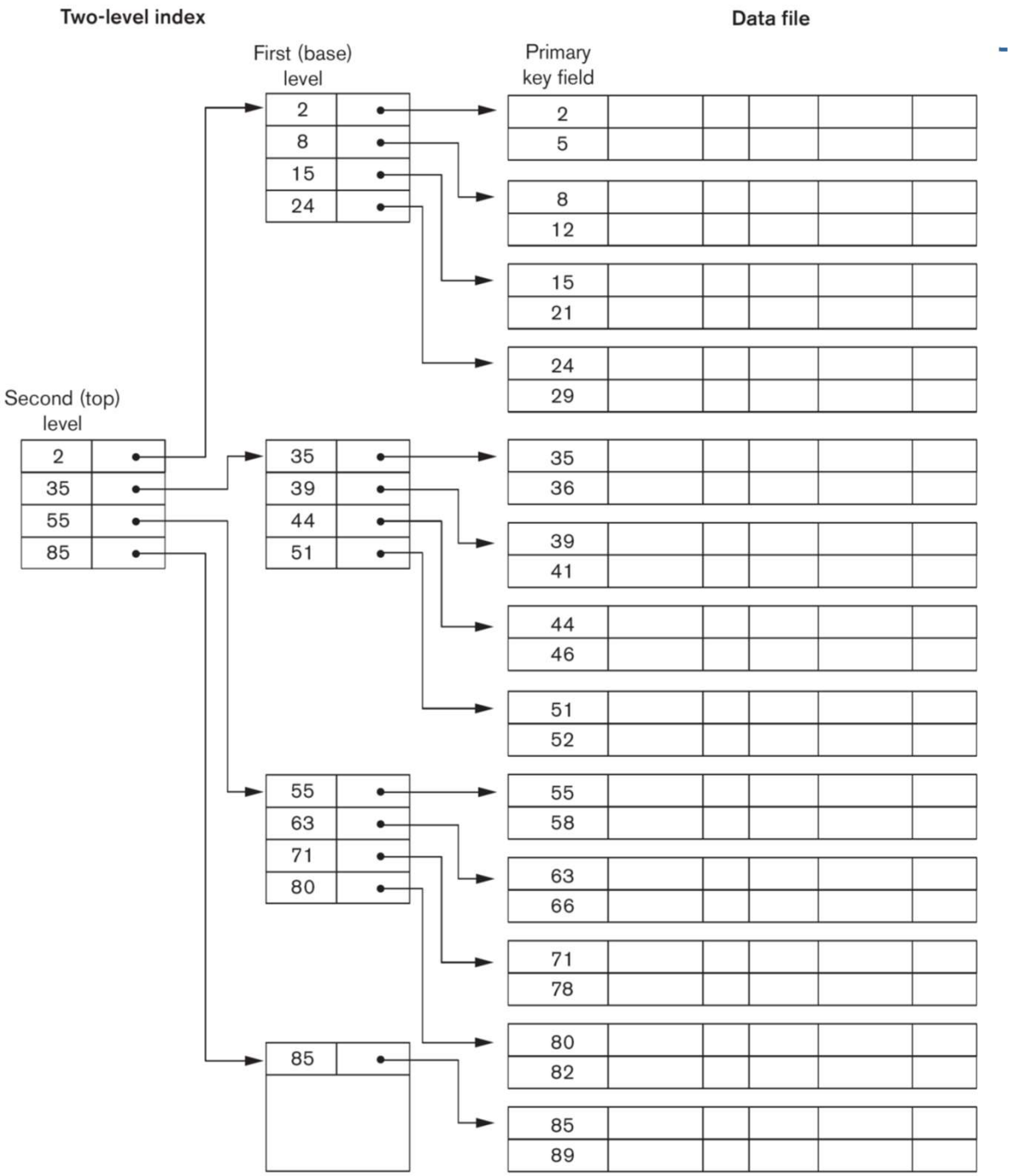
Type of Index	Number of (First-level) Index Entries	Dense or Nondense (Sparse)	Block Anchoring on the Data File
Primary	Number of blocks in data file	Nondense	Yes
Clustering	Number of distinct index field values	Nondense	Yes/no[a]
Secondary (key)	Number of records in data file	Dense	No
Secondary (nonkey)	Number of records[b] or number of distinct index field values[c]	Dense or Nondense	No

Multi-Level Indexes

- Because a single-level index is an ordered file, we can create a primary index *to the index itself*;
  - In this case, the original index file is called the *first-level index* and the index to the index is called the *second-level index*.
- We can repeat the process, creating a third, fourth, ..., top level until all entries of the *top level* fit in one disk block
- A multi-level index can be created for any type of first-level index (primary, secondary, clustering) as long as the first-level index consists of *more than one* disk block

A Two-Level Primary Index

Figure 18.6
A two-level primary index resembling ISAM (Indexed Sequential Access Method) organization.



Direct (Hashed) File

- Access directly any block of a known address.
- The Direct or Hashed File
  - Use hashing on a key to find the record
  - Directly access a block at a known address
  - Key field required for each record
- Direct files are often used where:
  - very rapid access is required,
  - fixed length records are used, and
  - records are always accessed one at a time.

Hashed Files

- Hashing for disk files is called External Hashing
- The file blocks are divided into M equal-sized buckets, numbered $bucket_0$, $bucket_1$, ..., $bucket_{M-1}$.
  - Typically, a bucket corresponds to one (or a fixed number of) disk block.
- One of the file fields is designated to be the hash key of the file
- The record with hash key value K is stored in bucket i, where i=h(K), and h is the hashing function.
- Search is very efficient on the hash key.
- Collisions occur when a new record hashes to a bucket that is already full.
  - An overflow file is kept for storing such records.
  - Overflow records that hash to each bucket can be linked together.

Hashed Files (cont.)

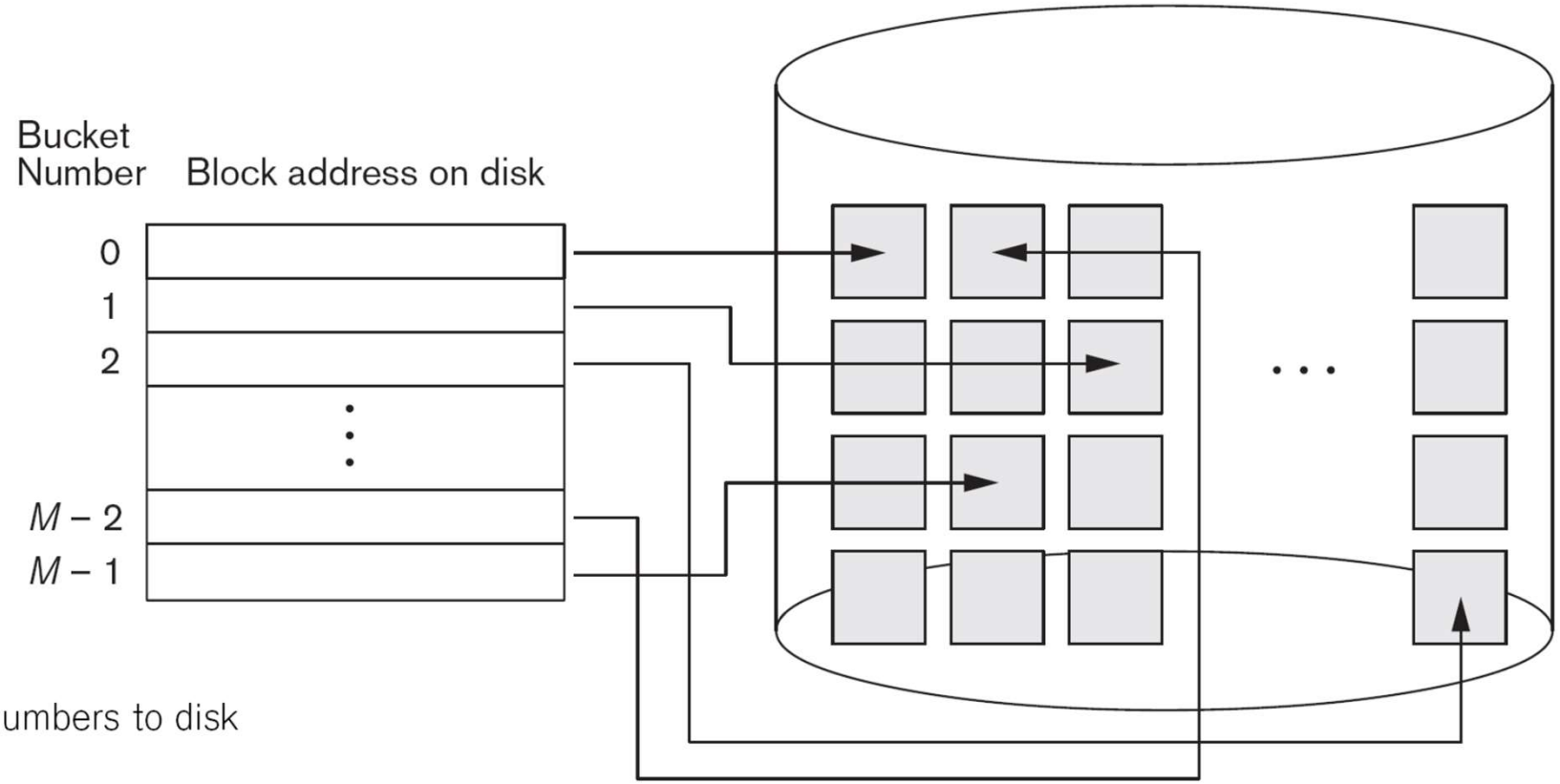


Figure 17.9
Matching bucket numbers to disk block addresses.

Hashed Files (cont.)

- There are numerous methods for collision resolution, including the following:
  - **Open addressing**: Proceeding from the occupied position specified by the hash address, the program checks the subsequent positions in order until an unused (empty) position is found.
  - **Chaining**: For this method, various overflow locations are kept, usually by extending the array with a number of overflow positions. In addition, a pointer field is added to each record location. A collision is resolved by placing the new record in an unused overflow location and setting the pointer of the occupied hash address location to the address of that overflow location.
  - **Multiple hashing**: The program applies a second hash function if the first results in a collision. If another collision results, the program uses open addressing or applies a third hash function and then uses open addressing if necessary.

Hashed Files - Overflow Handling

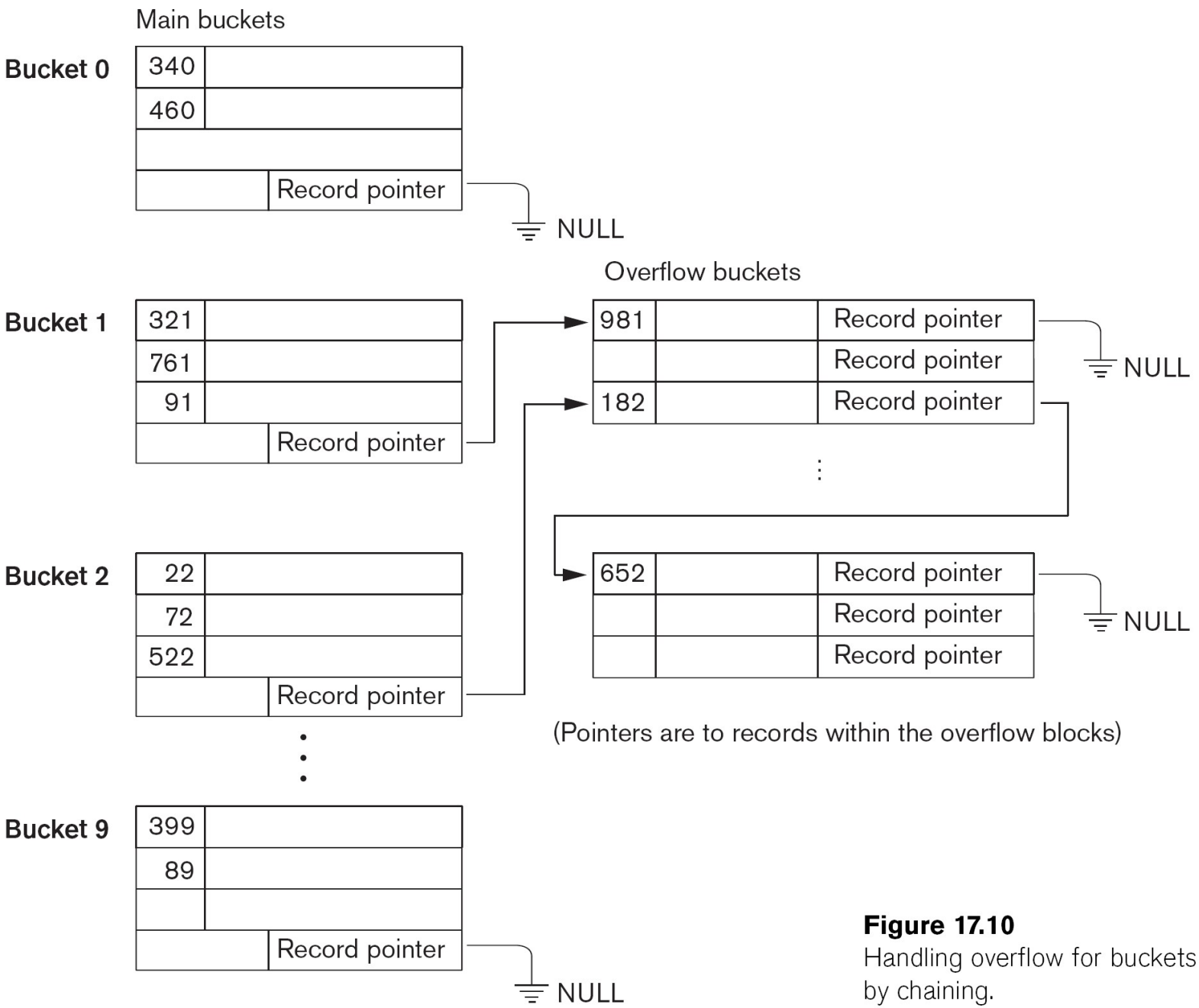


Figure 17.10
Handling overflow for buckets by chaining.

Hashed Files (cont.)

- To reduce overflow records, a hash file is typically kept 70-80% full.
- The hash function ***h*** should distribute the records uniformly among the buckets
  - Otherwise, search time will be increased because many overflow records will exist.
- Main disadvantages of static external hashing:
  - Fixed number of buckets M is a problem if the number of records in the file grows or shrinks.
  - Ordered access on the hash key is quite inefficient (requires sorting the records).

Performance

Table 12.1 Grades of Performance for Five Basic File Organizations [WIED87]

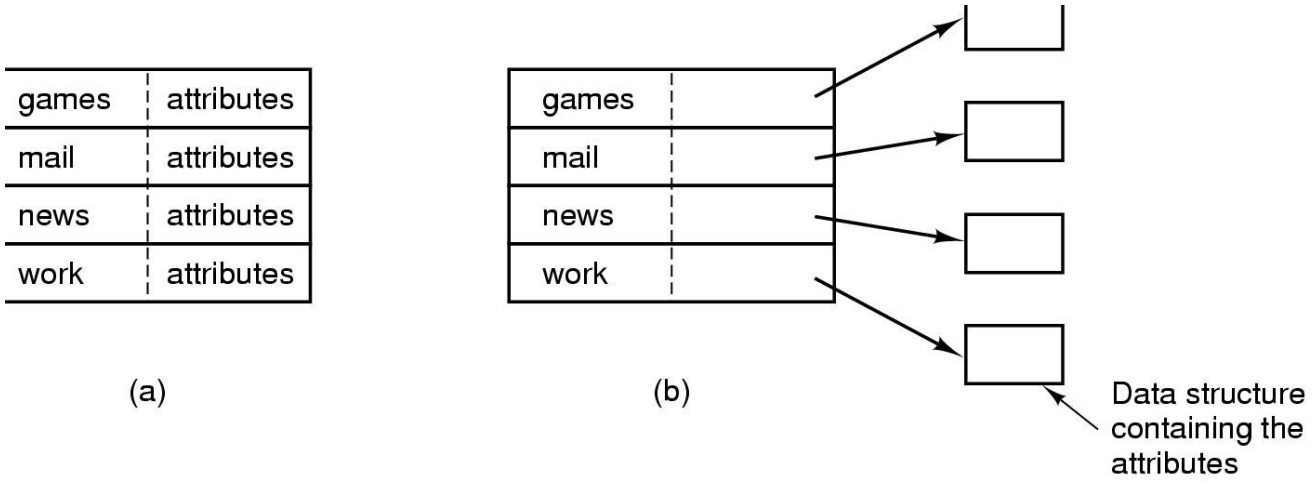
	Space		Update		Retrieval		
	Attributes		Record Size				
File Method	Variable	Fixed	Equal	Greater	Single record	Subset	Exhaustive
Pile	A	B	A	E	E	D	B
Sequential	F	A	D	F	F	D	A
Indexed sequential	F	B	B	D	B	D	B
Indexed	B	C	C	C	A	B	D
Hashed	F	B	B	F	B	F	E

A	=	Excellent, well suited to this purpose	$\approx O(r)$
B	=	Good	$\approx O(o \times r)$
C	=	Adequate	$\approx O(r \log n)$
D	=	Requires some extra effort	$\approx O(n)$
E	=	Possible with extreme effort	$\approx O(r \times n)$
F	=	Not reasonable for this purpose	$\approx O(n^{>1})$

where

r = size of the result
o = number of records that overflow
n = number of records in file

Directories



- Contains information about files
  - Names, Attributes, Ownership
  - Location
  - who is allowed to access the file
- Directory itself is a file owned by the operating system
- Provides mapping between file names and the files themselves

Directory Elements

- **Basic information**
  - File name
  - File type
  - File organisation (when different organizations are supported)
- **Address information**
  - Volume: device on which file is stored
  - Starting address
    - Block #, cylinder #, or other location id
  - Size used: current size of the file in bytes, words, or blocks
  - Size allocated: the maximum size of the file

Directory Elements (Contd)

- **Access Control Information**
  - Owner: the owner may be able to grant/deny access to other users and to change these privileges.
  - Access Information: May include the user's name and password for each authorized user.
  - Permitted Actions: Controls reading, writing, executing, transmitting over a network
- **Usage Information**
  - Date Created
  - Identity of Creator
  - Date Last Read Access
  - Identity of Last Reader
  - Date Last Modified
  - Identity of Last Modifier
  - Date of Last Backup
  - Current Usage : Current activity, locks, etc

Operations Performed on a Directory

- A directory system should support a number of operations including:
  - Search: When a user/ app references a file, the directory must be searched to find the entry corresponding to that file
  - Create files: an entry must be added to the directory
  - Deleting files: an entry must be removed from the directory
  - Listing directory: a listing of all files owned by that user, plus some of the attributes of each file
  - Updating directory: Because some file attributes are stored in the directory, a change in one of these attributes requires a change in the corresponding directory entry

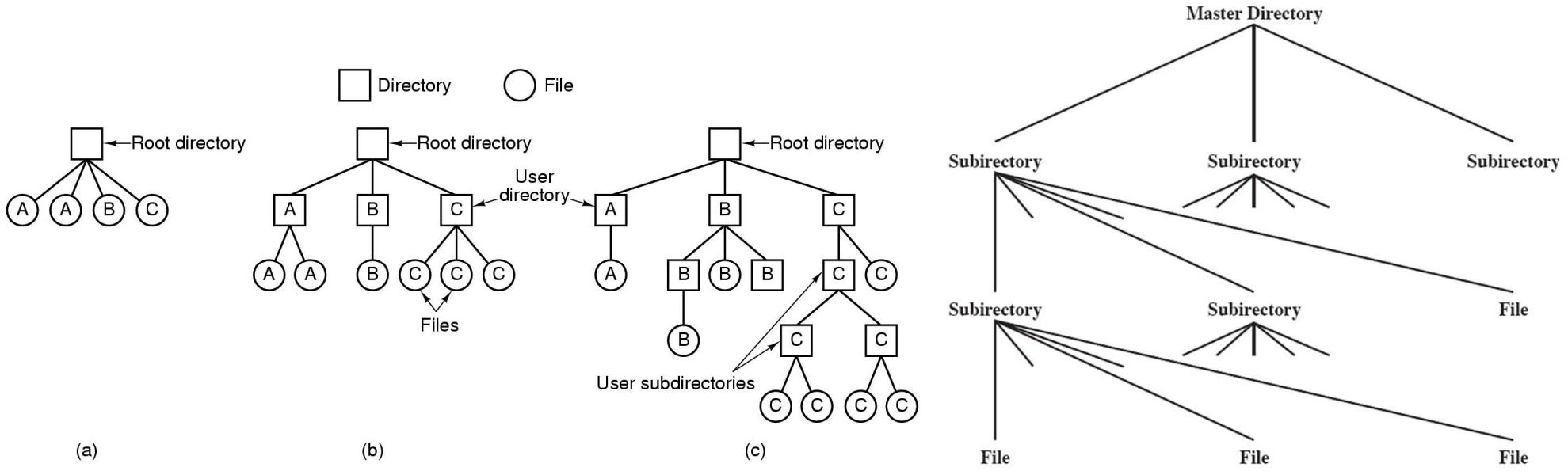
Directory Structure

- The method for storing the previous information varies widely between systems: could be in a header file reducing the size of the directory
- Simple Structure: list of entries, one for each file
  - Sequential file with the name of the file serving as the key
  - Provides no help in organizing the files
  - Forces user not to use the same name for two different files
- Two-Level Scheme
  - One directory for each user and a master directory
    - Master directory contains entry for each user
    - Provides address and access control information
  - Each user directory is a simple list of files for that user
    - Does not provide structure for collections of files

Directory Structure

Hierarchical, or Tree-Structured Directory

- Master directory with user directories underneath it
- Each user directory may have subdirectories and files as entries
- Names only unique in directory
- Each directory can be a sequential file, but hashed files are preferred to offer better performance in case directories contain a very large number of entries



Naming

- Users need to refer to a file by name
- Path - following set of directories from master directory to file
  - Example: /UserB/Word/UnitA/ABC
  - “/” often used to separate directories
  - File names not unique – only unique pathnames
- Working directory
  - Files are referenced relative to the working directory
  - Ex: working directory for user B is “Word,” then the pathname Unit_A/ABC

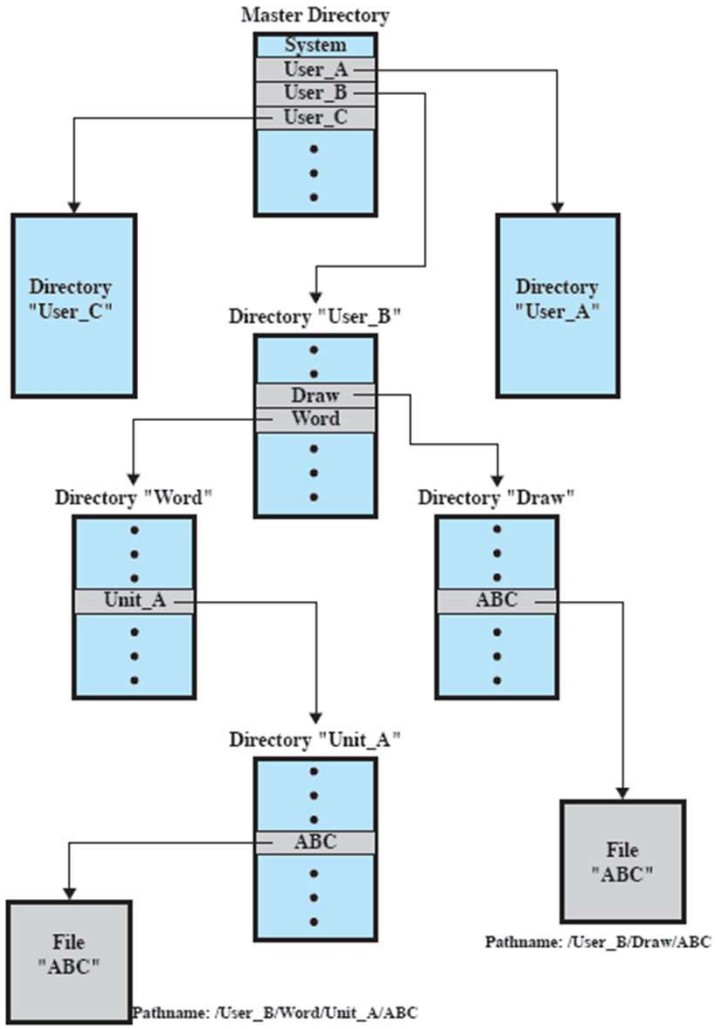


Figure 12.5 Example of Tree-Structured Directory

File Sharing

- In multiuser system, allow files to be shared among users
- Two issues arise
  - Access rights
  - Management of simultaneous access

File Sharing – Access Rights

- Users/groups are granted certain access rights to a file
- A wide range of access rights has been used.
  - None – Others don't know it exists
  - Knowledge – Know it is there and who the owner is
  - Execution – Able to run a program
  - Read – Look at or copy contents
  - Append – Add to but not modify data in file
  - Update – Modify/delete/add data
  - Change Protection – Grant rights to file
  - Deletion – Can delete file
- These rights can be considered to constitute a hierarchy, with each right implying those that precede it.
- Classes of Users: Right may be granted by Owner to
  - A specific user: allow different users to have distinct permissions
  - A group of users
  - The world (public files)

File Sharing – Simultaneous Access

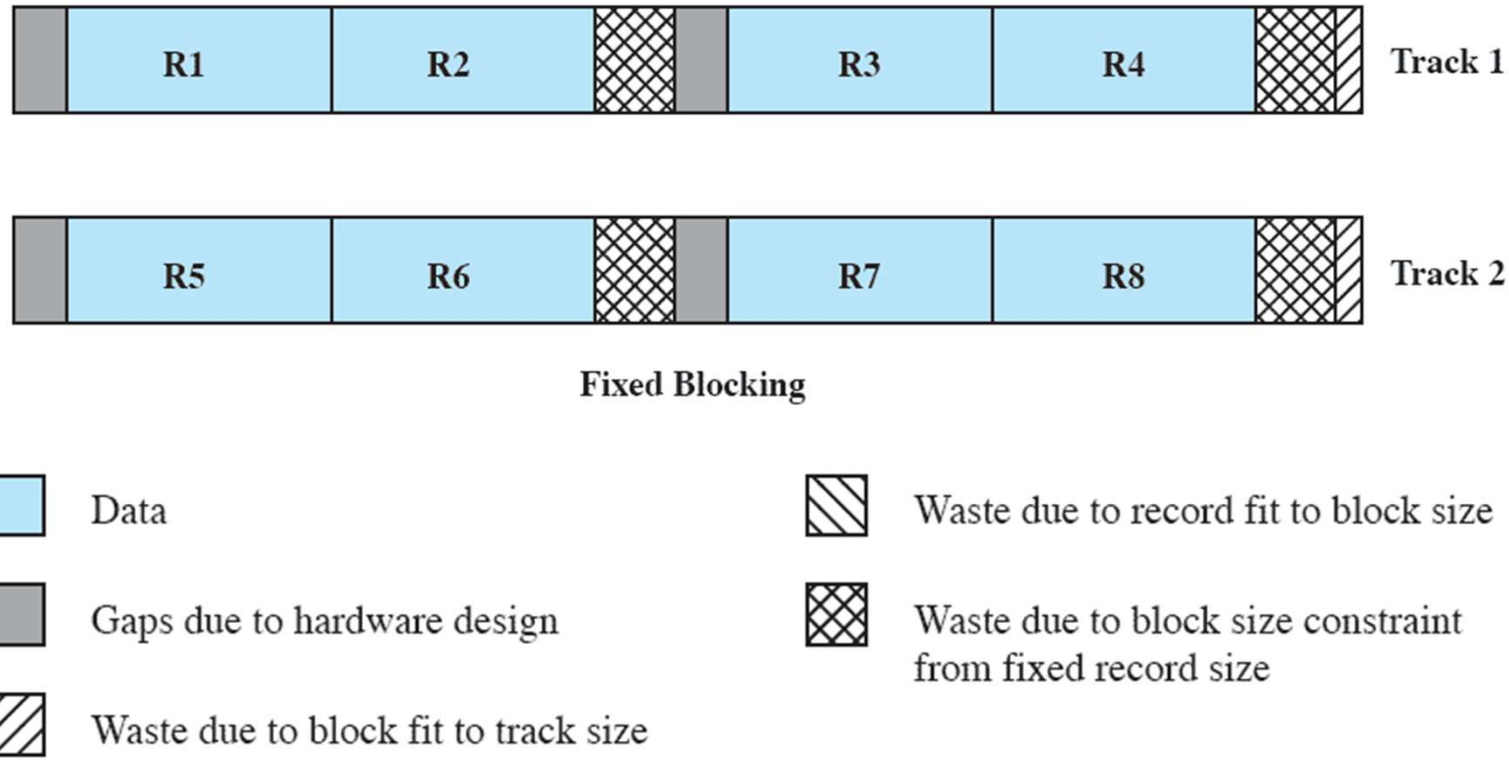
- Simultaneous Access
  - Multiple users may want to access or modify the same file e.g. Airline reservation database
  - OS and FMS must enforce discipline
    - Instance of reader/writer problem
    - Must address mutual exclusion and deadlock
    - Locking: Entire file vs. Records
      - Easier to lock entire file
      - Locking records allows more concurrency

Blocks and records

- Records are the logical unit of access of a structured file
- Blocks are the unit for I/O with secondary storage
- Records must be organized as blocks
- Block size: Usually fixed length
  - simplifies I/O, buffer allocation in main memory,
  - simplifies the organization of blocks on secondary storage
- Large vs Small blocks. Large blocks
  - ++ transfer more records per I/O operation, reduce access time
  - - - Wastes space/time if other records not used (random reads)
  - - - Require larger I/O buffers
- Three approaches are common
  - Fixed length blocking
  - Variable length spanned blocking
  - Variable-length unspanned blocking

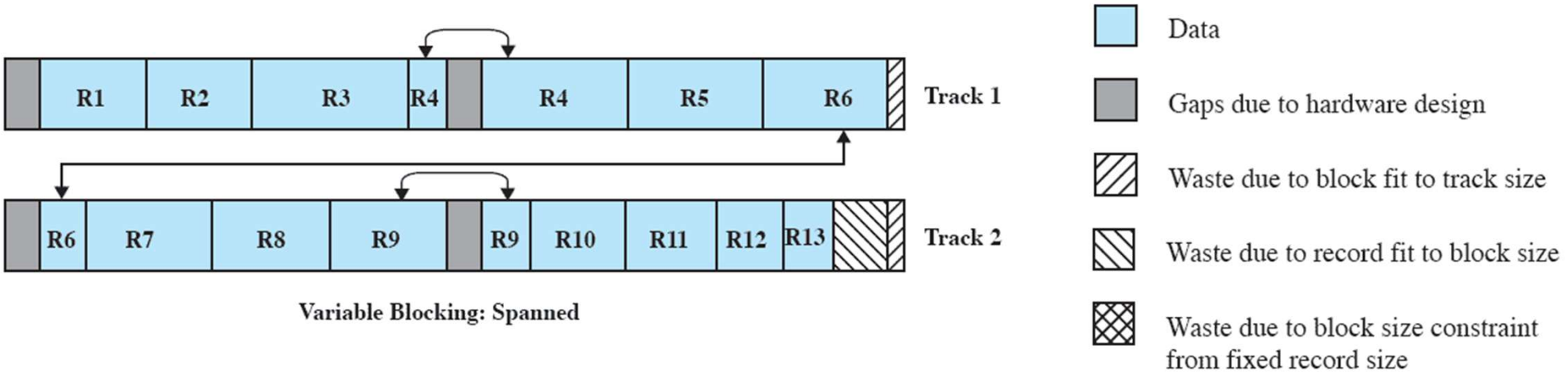
Fixed Blocking

- Fixed-length records are used
  - Integral # of records per block
  - Easy to find an arbitrary record
- Unused space at the end of a block is internal fragmentation



Variable Length - Spanned Blocking

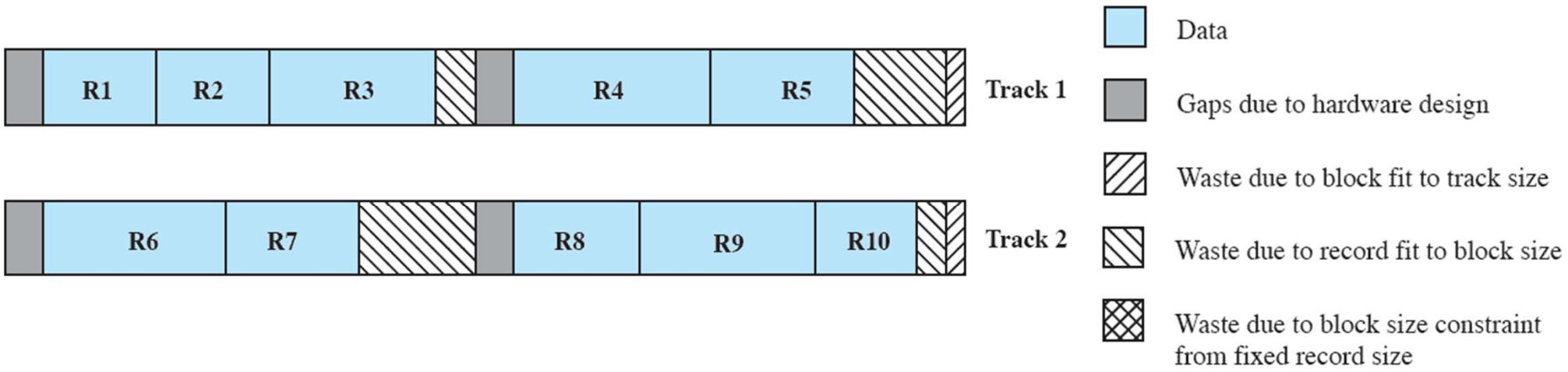
- Variable-length records are used
- ++ Records are packed into blocks with no unused space
- Some records may span multiple blocks
  - Continuation is indicated by a pointer to the successor block
  - Size of a record can be larger than a block ++
  - Have to read both blocks for that record - -
- Files are difficult to update - -
- Wastes space only at the end of the file



Variable Blocking: Spanned

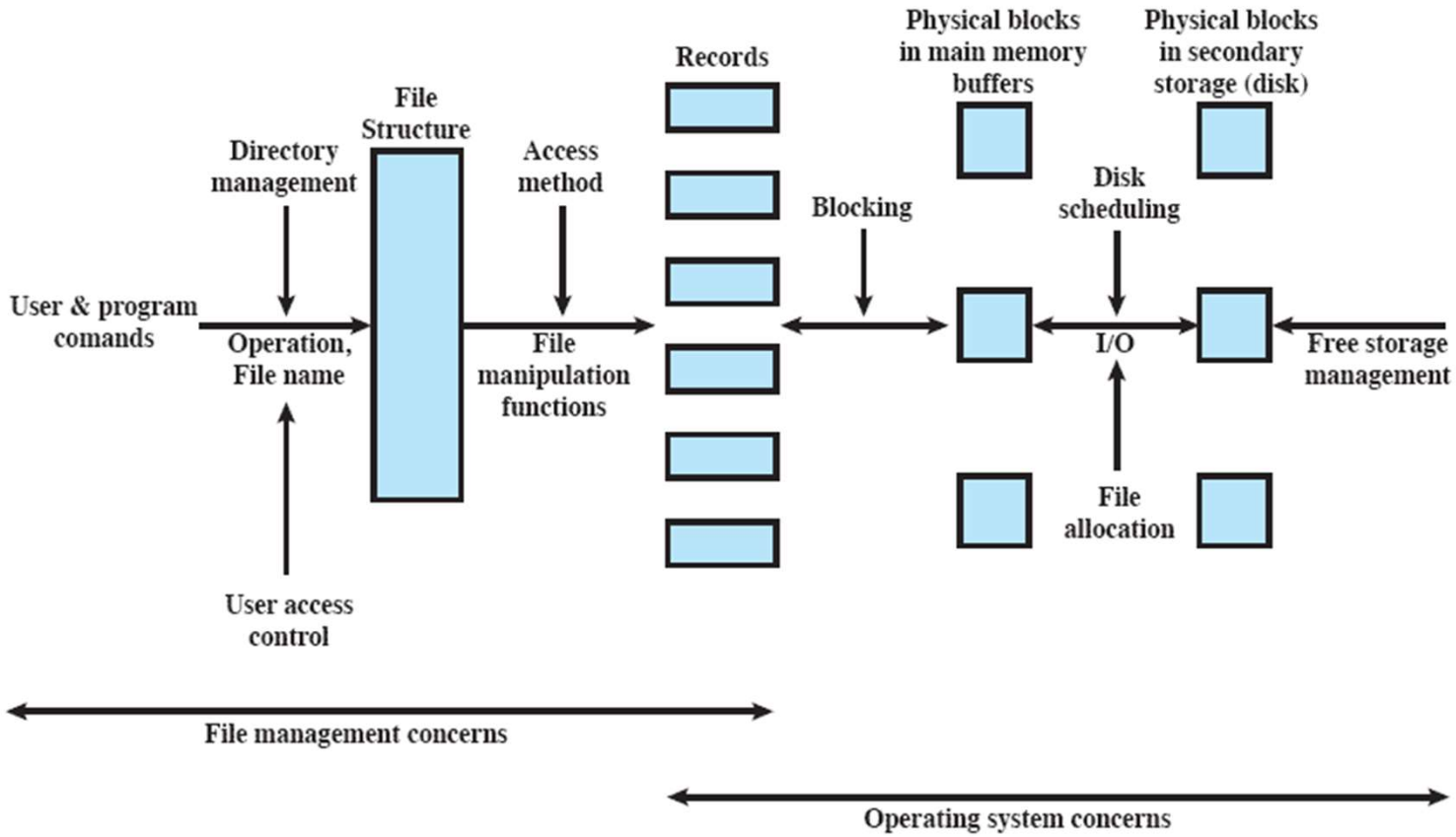
Variable-length: unspanned blocking

- Uses variable length records without spanning
- Wasted space in most blocks because of the inability to use the remainder of a block if the next record is larger than the remaining unused space.
- Limits the size of a record to the size of a block



Secondary Storage Management

- OS is responsible for allocating blocks to files
- Two related issues
  - Space on secondary storage must be allocated to files
  - Must keep track of the space available for allocation



File allocation issues

1. When a file is created – is the maximum space allocated at once?
2. Space is added to a file in contiguous 'portions'
   - What size should be the 'portion'?
3. What data structure should be used to keep track of the file portions?
   - Ex: File Allocation Table (FAT)

Preallocation vs Dynamic Allocation

- **Preallocation**
  - Need the maximum size for the file at the time of creation → contiguous
  - Difficult to reliably estimate the maximum potential size of the file
  - Tend to overestimate file size so as not to run out of space → wasted space
- **Dynamic allocation**
  - Get space (portions) as the file needs it
  - Files are often no longer contiguous

Portion size

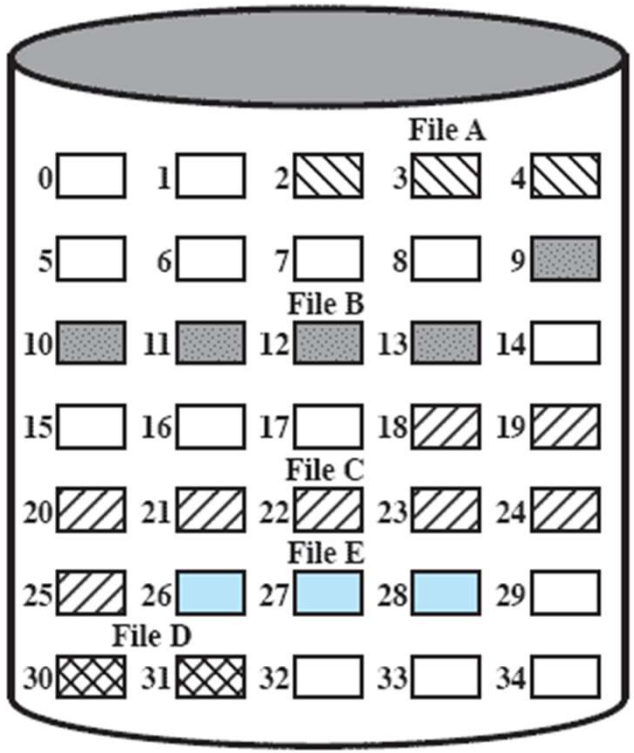
- Two extremes:
  - Portion large enough to hold entire file is allocated
  - Allocate space one block at a time
- **Tradeoffs**: single file efficiency vs system efficiency
  - Larger units are contiguous and increase performance
  - Lots of small units requires more space for allocation tables
  - Fixed-size portions simplifies the reallocation of space
  - Variable-sized units or small fixed-size units reduces wasted space
- **Variable-sized large contiguous portions**
  - Minimizes waste, file allocation tables are small
  - Have to deal with fragmentation
    - First-Fit, Best-Fit, Nearest-Fit allocation
- Blocks – Small fixed-size portions
  - Abandons contiguity, requires large tables
  - High flexibility, Allocate blocks as needed

File Allocation Method

- Three methods are in common use:
  - contiguous
  - chained, and
  - indexed

Contiguous Allocation

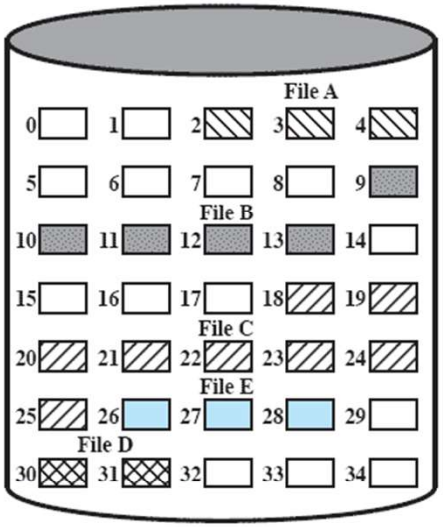
- Single set of blocks is allocated to a file at the time of creation
  - Pre-allocation strategy with variable-sized portions
- Only a single entry in the file allocation table
  - Starting block and length of the file
- Good performance (especially for sequential files: Multiple blocks can be read in at a time)
- Needs size of the file pre-declared
- External fragmentation will occur
  - Need to perform compaction



File Allocation Table

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

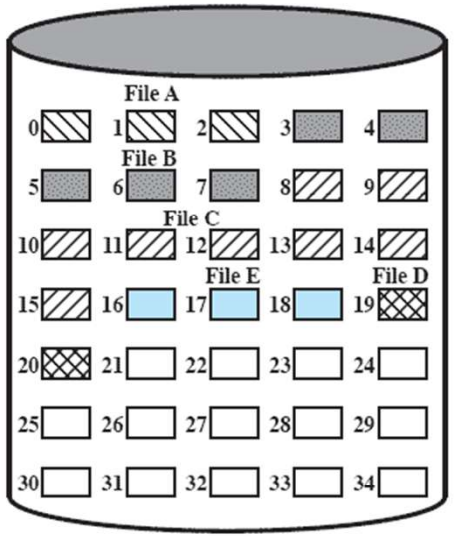
External fragmentation



File Allocation Table

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Figure 12.7 Contiguous File Allocation



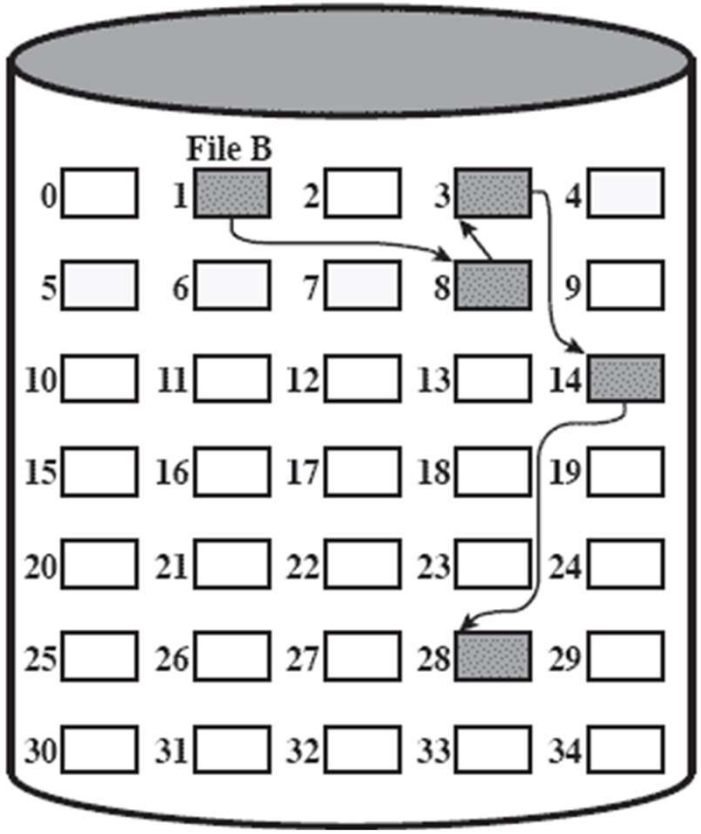
File Allocation Table

File Name	Start Block	Length
File A	0	3
File B	3	5
File C	8	8
File D	19	2
File E	16	3

Figure 12.8 Contiguous File Allocation (After Compaction)

Chained Allocation

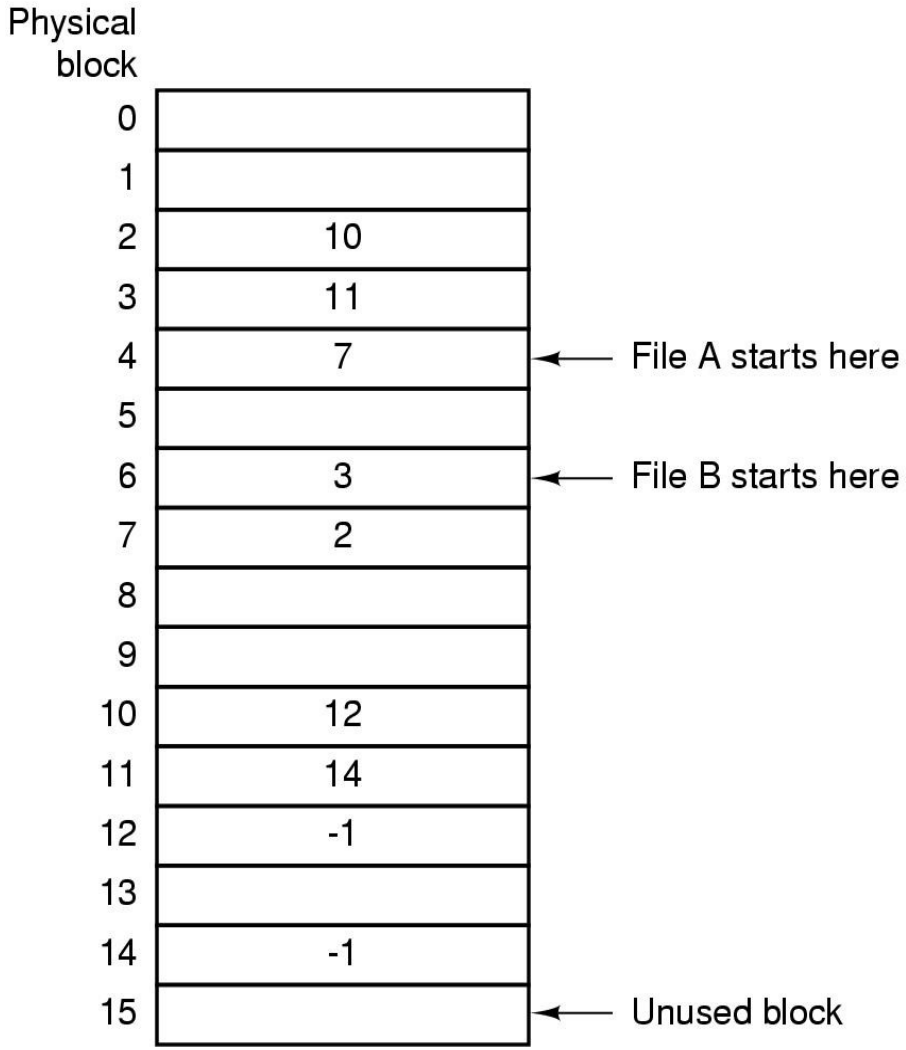
- Allocation on basis of individual block
- Each block contains a pointer to the next block in the chain
- Only single entry in the file allocation table
  - Starting block and length of file
- ++ No external fragmentation
- ++ No need for preallocation
- ++ Selection of blocks is easy: any free block can be added to the chain
- -- To access a specific block – trace thru the chain to the desired block
- -- No accommodation for locality

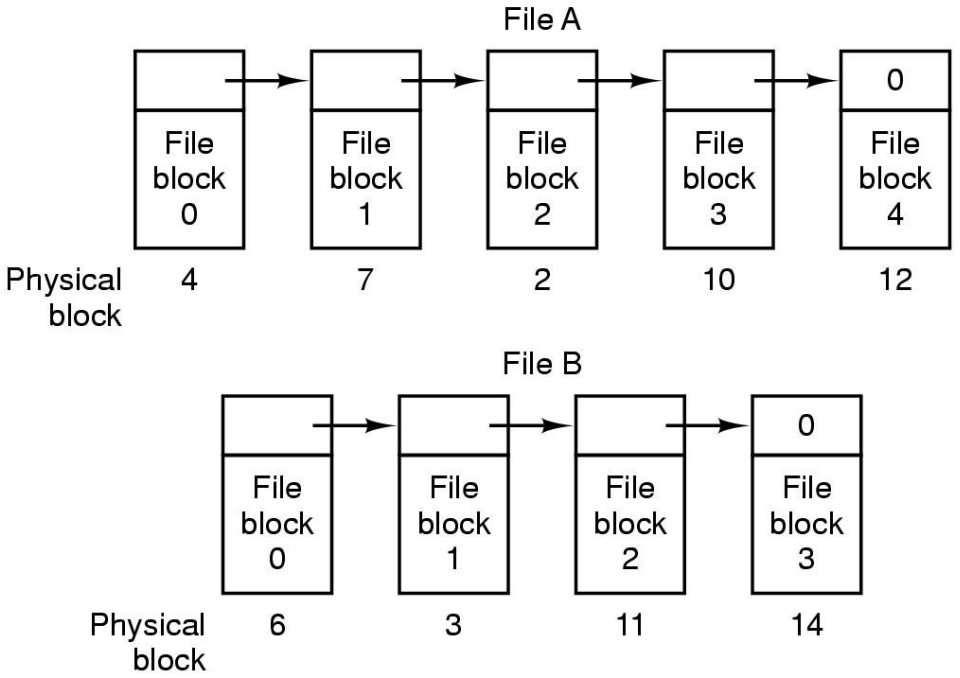


File Allocation Table

File Name	Start Block	Length
...	...	...
File B	1	5
...	...	...

Alternative FAT for Chained Allocation

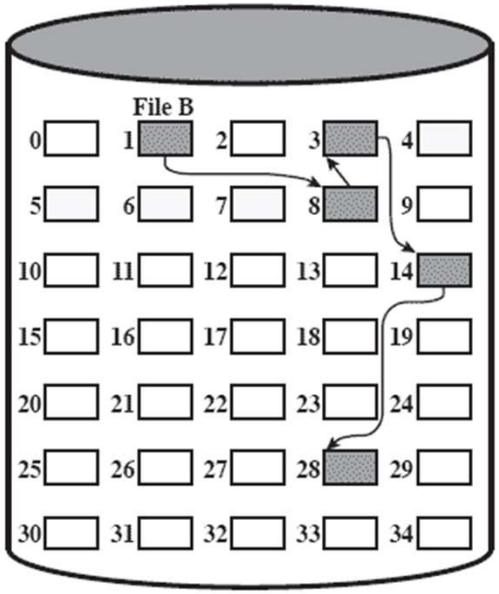




Keep the list links in a separate table in memory

Chained Allocation Consolidation

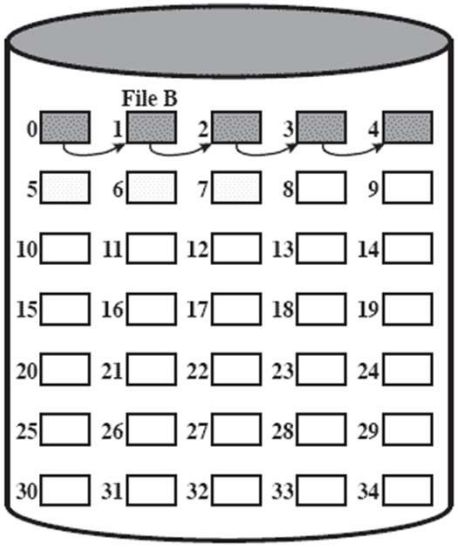
no accommodation of
the principle of locality



File Allocation Table

File Name	Start Block	Length
• • •	• • •	• • •
File B	1	5
• • •	• • •	• • •

Figure 12.9 Chained Allocation

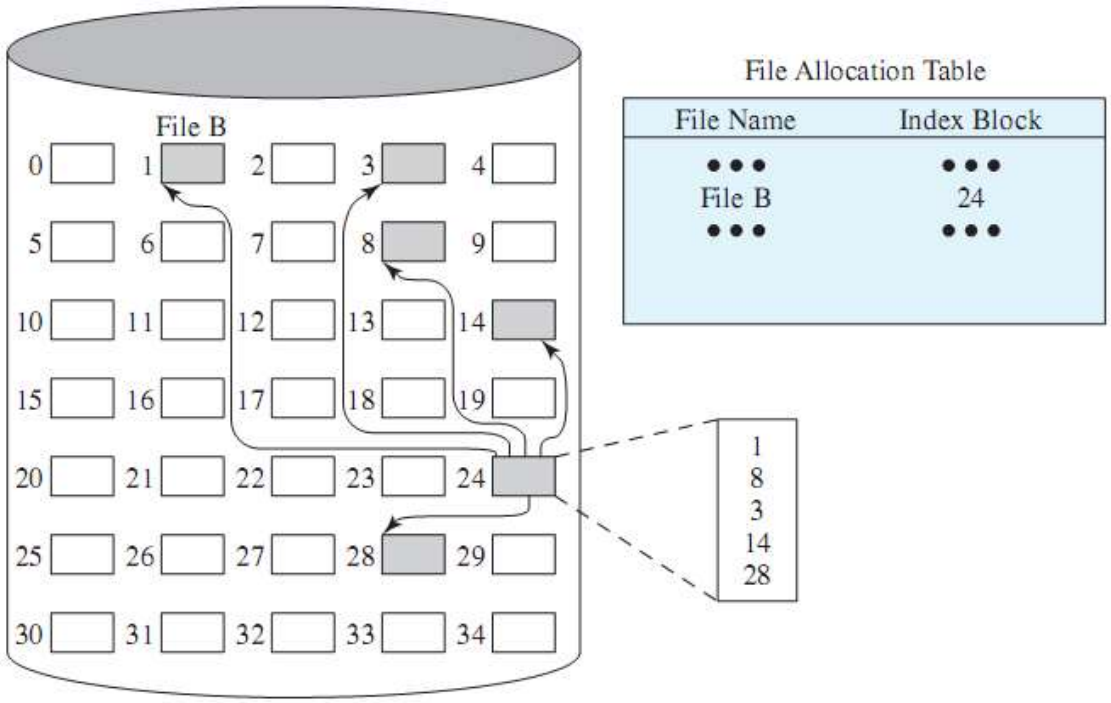


File Allocation Table

File Name	Start Block	Length
• • •	• • •	• • •
File B	0	5
• • •	• • •	• • •

Figure 12.10 Chained Allocation (After Consolidation)

Indexed Allocation



- File allocation table has a pointer to an index for each file
- Index block has pointers to the data blocks in the file
- the file index for a file is kept in a separate block
- Supports both sequential and direct access to a file
- Most common form of allocation Unix uses a variation on this
- Allocation may be either
  - Fixed size blocks or - eliminates external fragmentation
  - Variable sized portions – improving locality
- Both cases require occasional consolidation

Indexed Allocation with Variable Length Portions

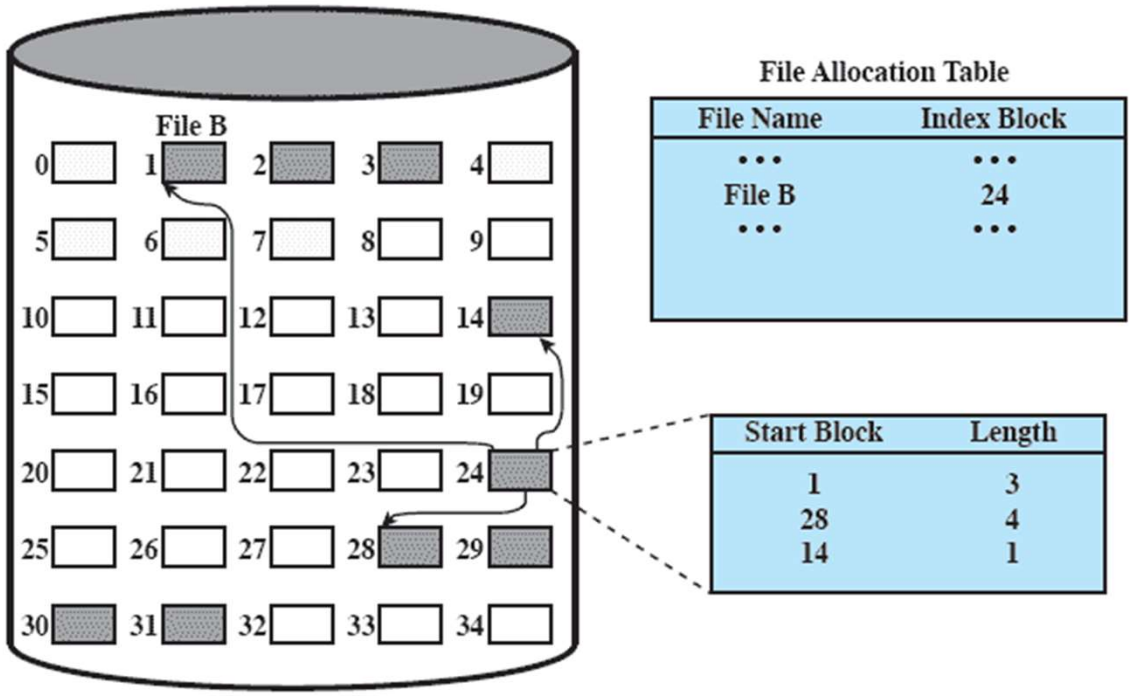


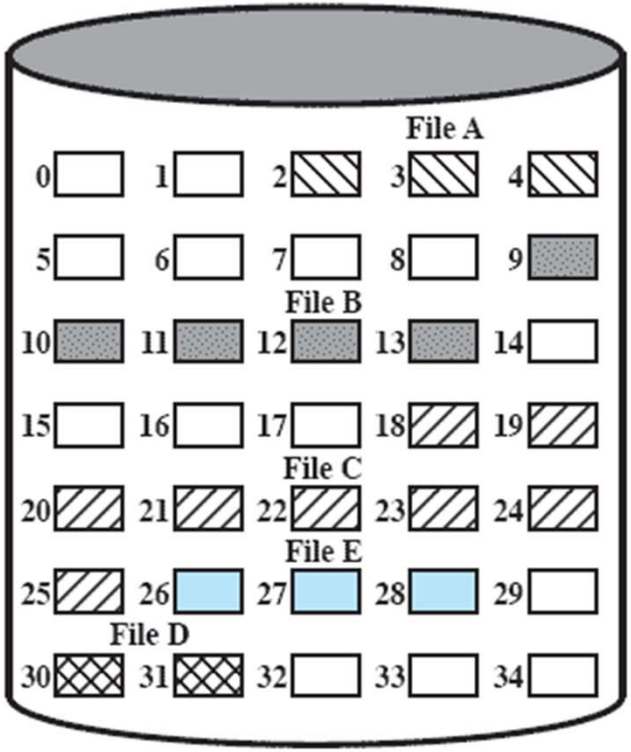
Figure 12.12 Indexed Allocation with Variable-Length Portions

Free Space Management

- Just as allocated space must be managed, so must the unallocated space
- To perform file allocation, we need to know which blocks are available.
- We need a Disk Allocation Table in addition to a File Allocation Table

Bit Tables

- Uses a vector containing one bit for each block on the disk.
  - **0** corresponds to a free block,
  - **1** corresponds to a block in use.
- **Advantages:**
  - Works well with any file allocation method
  - Easy to find one or a contiguous group of free blocks
  - As small as possible
- **Disadvantages**
  - Still table can be sizable to be placed in main memory: for a 16GB disk with 512-byte blocks, the bit table needs 4MB
  - Even if in main memory, exhaustive search can slow down performance



00111000011111000011111111111011000

$$\frac{\text{disk size in bytes}}{8 \times \text{file system block size}}$$

Chained Free Portions

- The free portions may be chained together by using a pointer and length value in each free portion.

++ Negligible space overhead – no need for DAT, pointer at the beginning of the chain and the length of the portion

++ Suited to all file allocation methods

  - If allocation is a block at a time, choose the free block at the head of the chain and adjust the first pointer or length value
  - If allocation is by variable-length portion, a first-fit algorithm may be used: The headers from the portions are fetched one at a time to determine the next suitable free portion in the chain. Again, pointer and length values are adjusted.

-- Leads to fragmentation, many portions become single block long

-- with allocating per block, needs read the block to get the pointer to the new first free block before writing data to that block -- slow file creation where many individual blocks need to be allocated

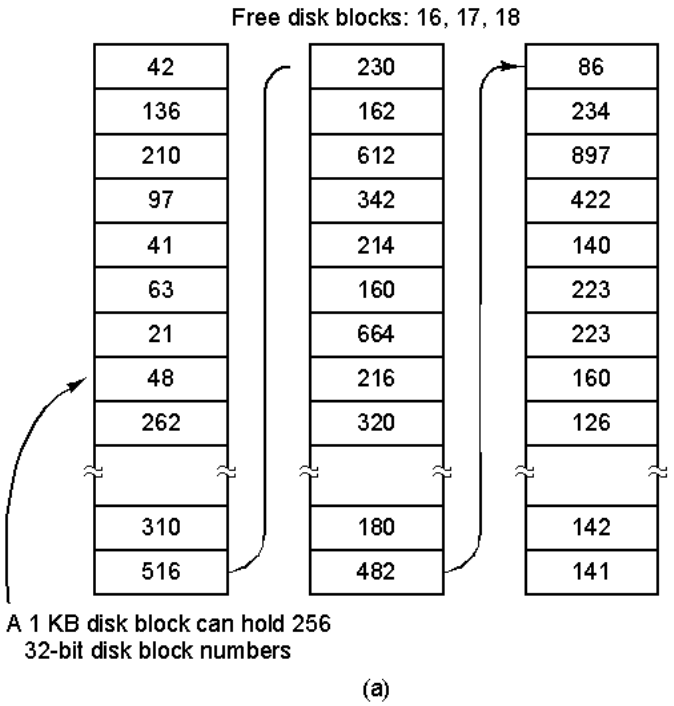
-- deleting highly fragmented files is very time consuming

Indexing

- Treats free space as a file and uses an index table as it would for file allocation
- For efficiency, the index should be on the basis of variable-size portions rather than blocks.
  - Thus, there is one entry in the table for every free portion on the disk.
- This approach provides efficient support for all of the file allocation methods.

Free Block List

- Each block is assigned a number sequentially
  - the list of the numbers of all free blocks is maintained in a reserved portion of the disk
- Can use FIFO order
- May have a background process that works to facilitate contiguous allocation
- Depending on the size of the disk, either 24 or 32 bits will be needed to store a single block number, so the size of the free block list is 24 or 32 times the size of the corresponding bit table and thus must be stored on disk rather than in main memory.
- Can treat it as a stack and re-allocate recently freed blocks – only the last few blocks need to be kept in memory



UNIX File Management

- Six types of files
  - Regular, or ordinary: information from user, application, or system utility
  - Directory: list of file names with pointer to index nodes (inodes)
  - Special – used to access peripheral devices (terminals or printers)
  - Named Pipes – I/O between processes
  - Links: alternative file name for an existing file
  - Symbolic links a data file that contains the name of the file it is linked to

Inodes

- Index node: a Control structure that contains key information for a particular file.
- Several filenames may be associated with a single inode
  - But an active inode is associated with only one file, and
  - Each file is controlled by only one inode
- The attributes of the file as well as its permissions and other control information are stored in the inode.

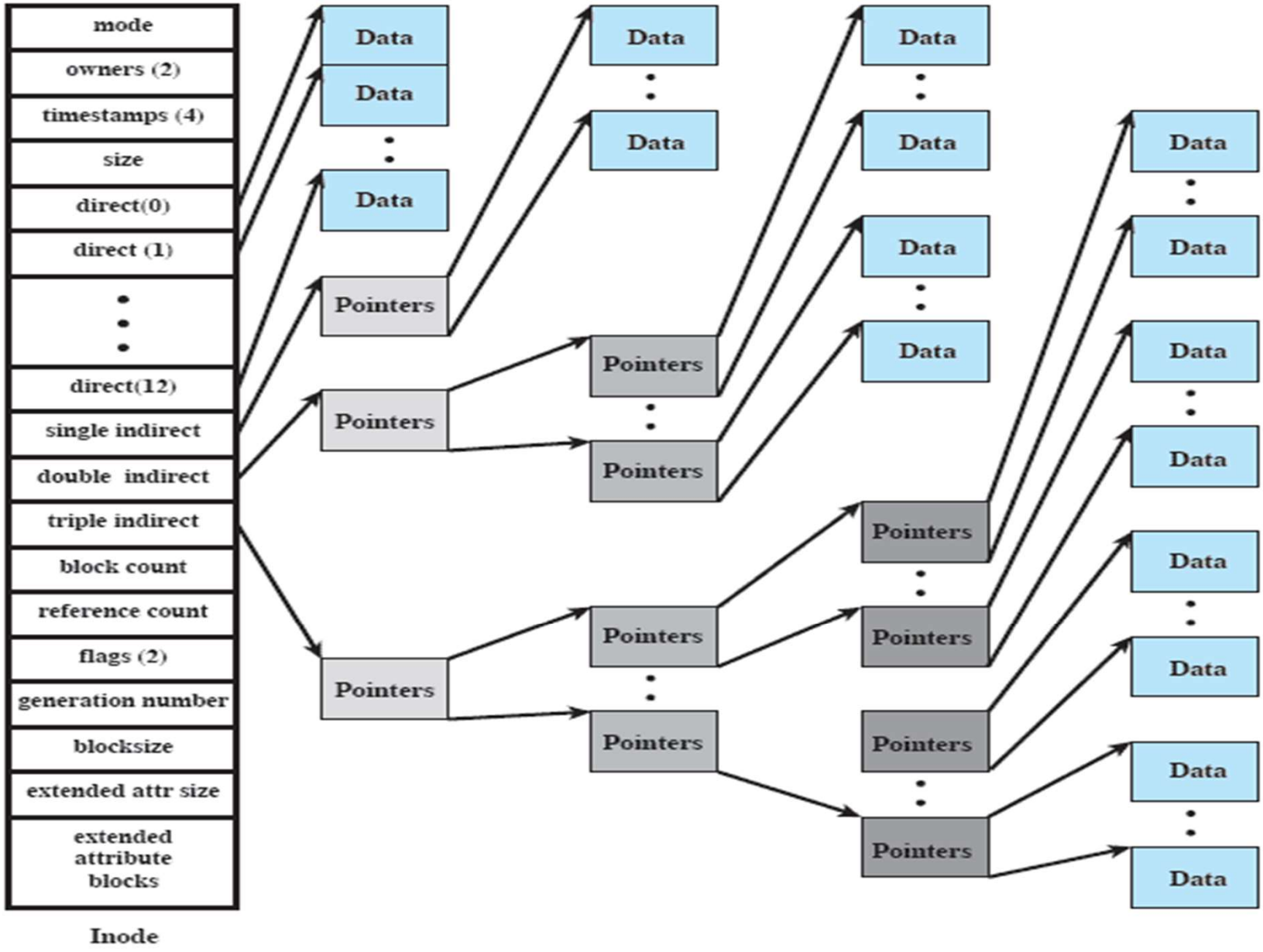
Free BSD Inodes include

- The type and access mode of the file
- The file's owner and group-access identifiers
- Creation time, last read/write time
- File size
- Sequence of block pointers
- Number of blocks and Number of directory entries
- Kernel and user setable flags
- Generation number for the file
- Size of Extended attribute information
- Zero or more extended attribute entries

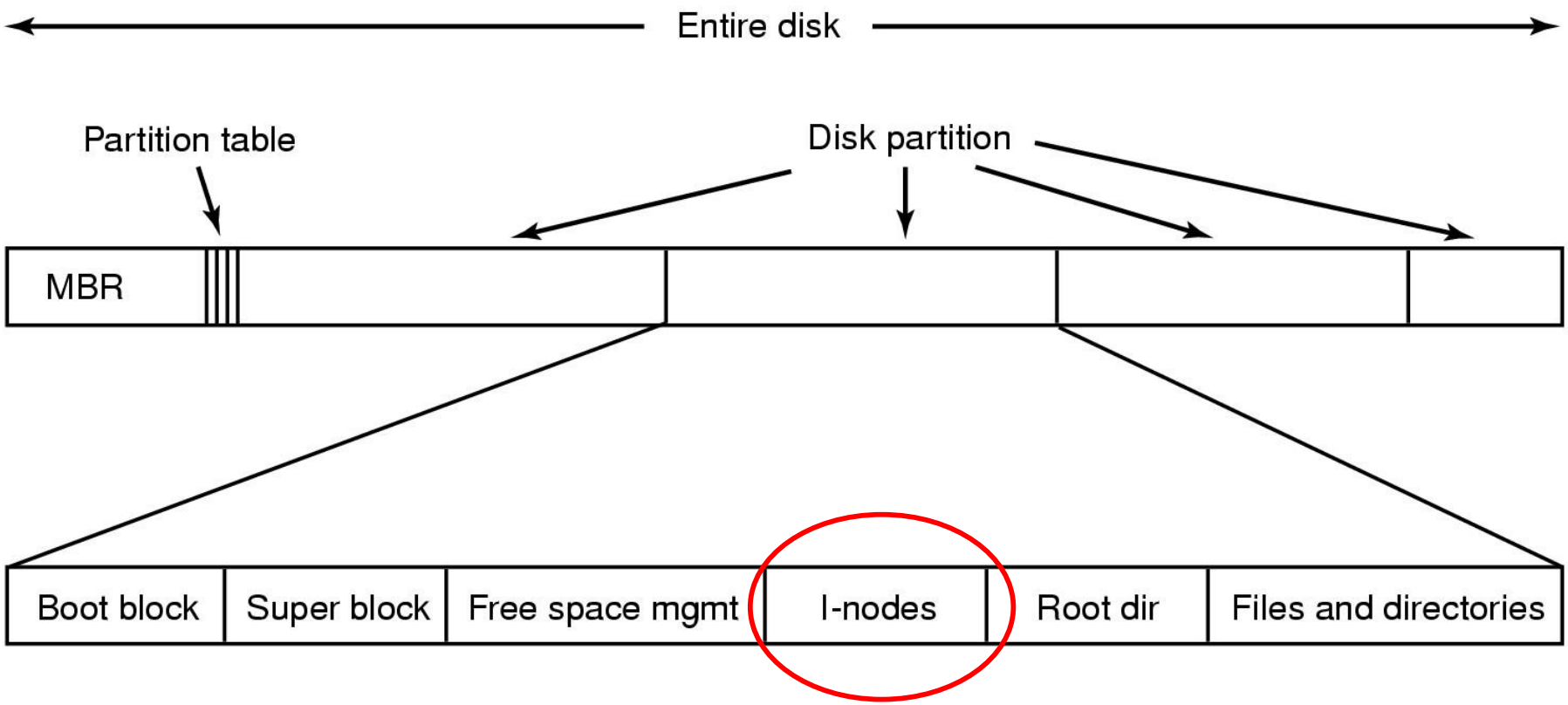
File Allocation

- File allocation is done on a block basis.
- Allocation is dynamic
  - Blocks may not be contiguous
- Index method keeps track of files
  - Part of index stored in the file inode.
- Inode includes a number of direct pointers and three indirect pointers
  - The first 10 addresses in an inode point to the first 10 blocks of a file
  - The 11th address points to a block containing the next portion of the index (single indirect block)
  - The 12th address points to a block of addresses that point to additional single indirect blocks (double indirect block)
  - The 13th (and final) address points to a triple indirect block that is a third level of indexing.

FreeBSD Inode and File Structure



Disk Organization



UNIX Directories and Inodes

- Directories are files containing:
  - a list of filenames
  - Pointers to inodes

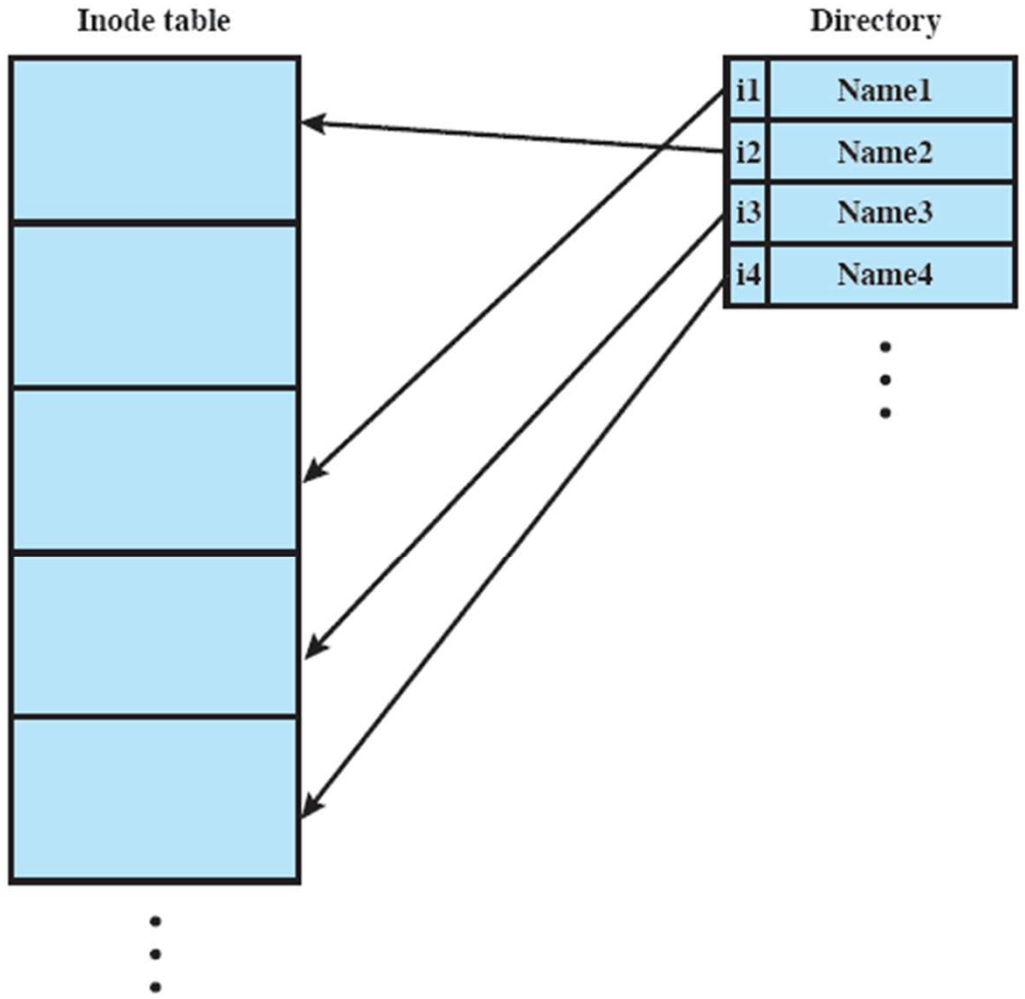
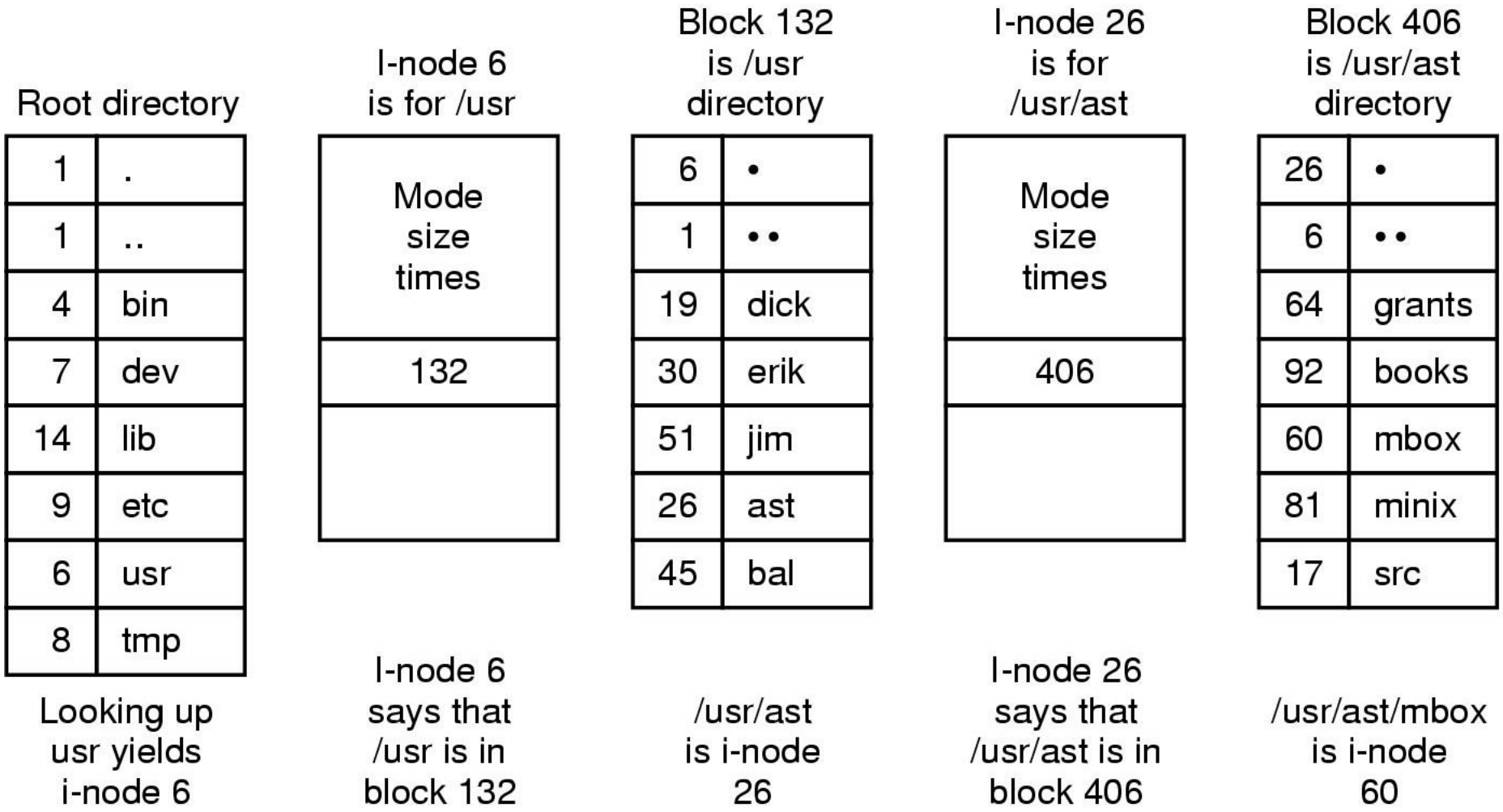


Figure 12.15 UNIX Directories and Inodes

The steps in looking up */usr/ast/mbox*



Windows File System

- Key features of NTFS (New Technology File System)
  - Large disks and large files
  - Multiple data streams
  - Journaling – Log file to note changes
  - Compression and Encryption
  - Recoverability
    - Transaction processing model: changes treated as atomic actions
  - Security

NTFS Volume and File Structure

- Sector
  - The smallest physical storage unit on the disk
  - Almost always 512 bytes
- Cluster
  - One or more contiguous sectors
  - Set of 1 to 128 sectors
  - Fundamental allocation unit
  - Helps handle sector sizes other than 512 bytes
  - Default cluster size depends on disk size (table next slide)
- Volume
  - Logical partition on a disk
  - May be all or part of a single disk
  - With RAID, may span several disks
  - Max size is 2^{64} bytes

Efficient with Large Files

Volume Size	Sectors per Cluster	Cluster Size
512Mbyte	1	512bytes
512Mbyte – 1 Gbyte	2	1K
1–2 Gbyte	4	2K
2–4 Gbyte	8	4K
4–8 Gbyte	16	8K
8–16 Gbyte	32	16K
16–32 Gbyte	64	32K
>32Gbyte	128	64K

NTFS Volume Layout

- Partition Boot Sector – first few sectors on any volume
  - Info on volume layout, file system structure, boot startup
- Master File Table – info on all files and directories
  - Table of 1024-byte rows
  - Each row describe a file and its attributes
- System Files
  - Log file: list of transactions used for NTFS recoverability
  - Cluster bit map
  - Attribute definition table
- Files

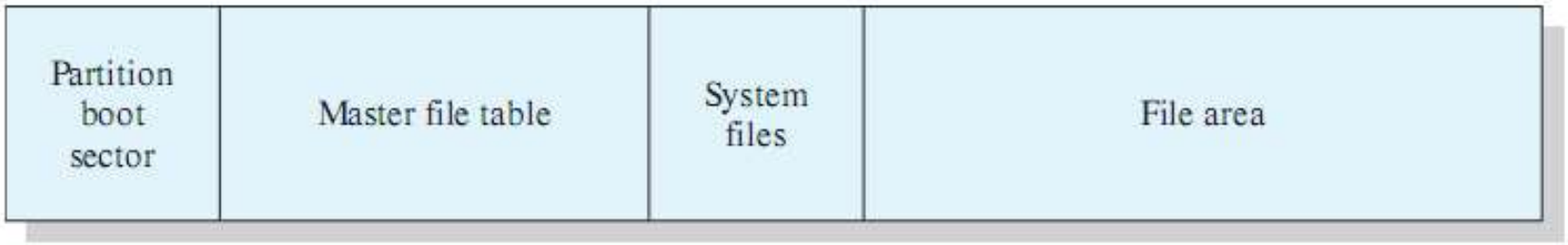
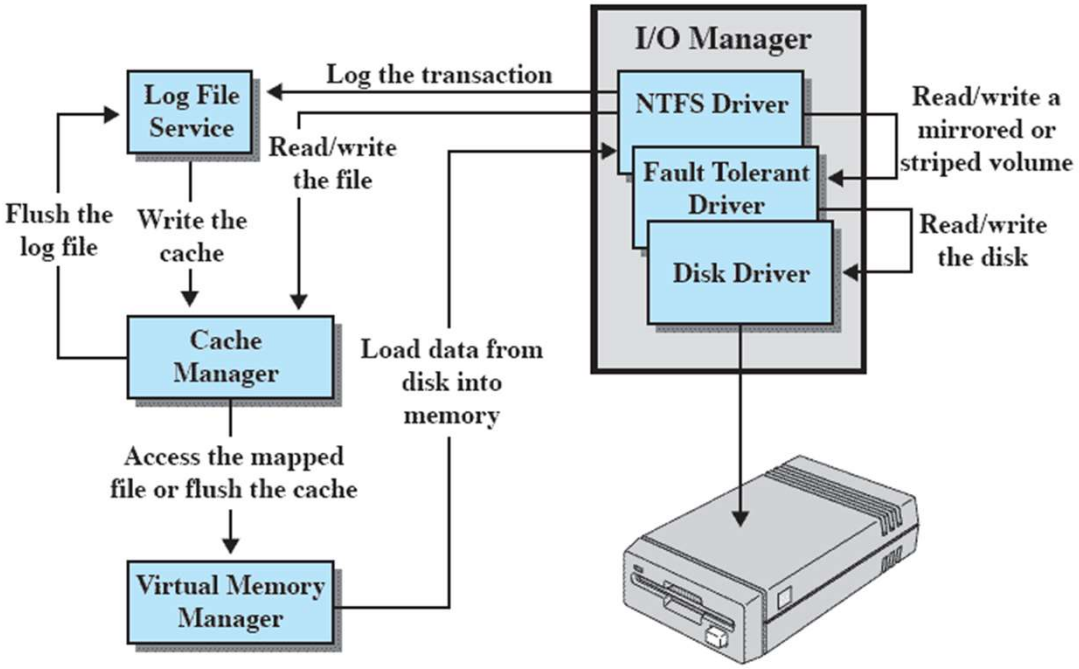


Figure 12.19 NTFS Volume Layout

Windows NTFS Components

- I/O manager
  - handles basic open, close, read, write, software RAID
- Log file service:
  - Keeps a log of disk writes
  - recover an NTFS-formatted volume in case of a crash
- Cache manager
  - Optimize disk I/O by lazy write and lazy commit
- Virtual memory manager
  - maps file references to virtual memory references
  - accesses cached files



Emphasis is recovery of system metadata, not user data